

UNIVERSITY OF MORATUWA

Faculty of Engineering



Department of Electronic and Telecommunication Engineering

EN 4020: Advanced Digital Systems

Assignment 1

Serial Bus Design

T.T.N Bahavan – 160050H

A.M Thathsara – 160626F

This report is submitted in partial fulfilment of the requirements
for the module EN 4020: Advanced Digital Systems

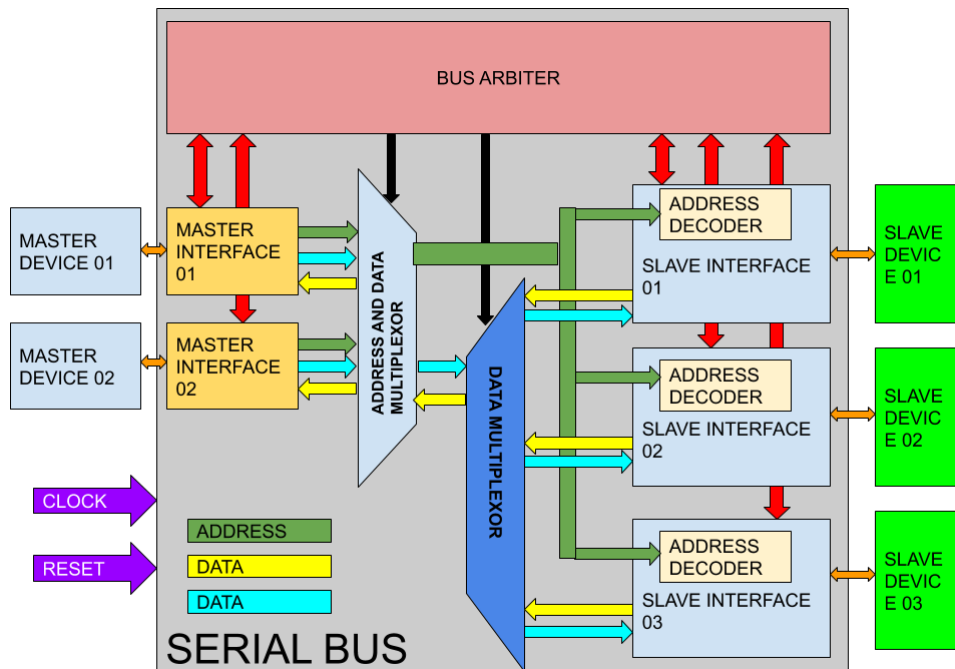
December 2020

Introduction (System Overview)

We present a fully integrated RTL code and self-checking testbench for a Serial Bus with a centralized arbiter, 2 Masters and 3 Slaves. The RTL is implemented with POSEDGE clocks and active-low asynchronous reset. The System can do Split transactions. The System consists of a Bus arbiter for priority level arbitration amongst the masters. It also has an address decoder for address mapping and slave selecting.

Design Terminology. (Please Read)

Overall Design



The **master device** is a device (like a processor) which wants to do a transaction. It attempts to access the resources a **slave device** like a hard drive has via these transactions. However, when you have multiple masters and multiple slaves, it is important to coordinate all the transactions. The master device interfaces the System Serial Bus using the **Master/Master Interface**. Throughout the report we will be referring this **master interface** as the **master**. The slave device interfaces the System Serial Bus using the **Slave/Slave Interface**. Throughout the report we will be referring this **Slave interface** as the **Slave**.

An Address Decoder is a module used to identify which slave the master is referring to. Commonly slave devices can be separate devices however the master might not know if is attached to slave1 or slave2 etc. The address decoder is built into the Slave interface. This is a design choice to simplify the overall architecture. It also gives great flexibility for the slave device as they can be connected in any slave port. It also allows scalability. The address decoder is self-selecting. It dramatically reduces the no. Of clock cycles taken for a transaction because the address is only transferred once.

Transactions

A Transaction is the transfer of data from one point of the hardware to another point. A Master **initiates** a transaction while a slave **responds** to that. An **address** is a location in the slave device which contains data or a place where data can be written.

A transaction in our design has two phases: **Address Transfer phase** and A **Data Transfer phase**. The address transfer phase sends information to the address decoders in the following format about the address and the type of

operation (read or write) to be performed in that location.

<i>1</i>	<i>1</i>	<i>0</i>	<i>0101010110</i>
<i>Start_bit (always one)</i>	<i>Read_bit</i>	<i>Write_bit (~ of the read bit)</i>	<i>Address</i>

Data phase transfers the write/read data between the master and slave. The data phase information in the following format.

<i>1</i>	<i>0110101010101</i>
<i>Start_bit (always one)</i>	<i>Data</i>

Split Transactions

Split transactions improve the overall utilization of the bus by separating (or splitting) the address phase and the data phase if the slave device is busy. A slave device maybe busy to do any read or write operation because of its internal operations

If a split transaction occurs, we say that the slave/master is '**split**' or in a '**split state**'. What this means is that the master will be blocked from using the bus to do any transactions until its current split transaction is over. The slave will also become unavailable for any transactions from other masters until the split transaction is over. Once the transaction which was split is completed, the master can continue doing other transactions and the slave will become available.

Bus Arbiter

Description and State Diagrams

We designed a fully centralized bus arbiter, which can control the flow of all the transactions.

Because the bus can only be used by one master at a time, it is important to assign priority levels for the masters. The priority level of each master is determined by the port it is connected to the arbiter. If a master is requesting access, the arbiter will grant access to the master for it to complete its transaction. If multiple masters are requesting access, the arbiter will grant access to the master with the highest priority and ignore the other masters. Other masters will be attended once the higher priority master's transaction is over.

In some cases, the slave might be busy to complete the current transaction. This might be because the read or write operation cannot be performed at the given time because the slave device connected to the slave is busy.

The arbiter will be at an idle state when both or one of the masters prompt the arbiter for access. Both masters will request access via the *requestIn* port. Once the arbiter receives the signal, it will first grant access via the *grantAccessOut* port to the higher priority master and ignore the other one. The Arbiter will go to a GRANT_STATE and wait.

During this process, the master will send out its address phase data to all the address decoders of the three slaves. If the address is within the specific range of an address decoder, that specific address decoder pulls up its relevant *slaveHitIn* wire. The other address decoders will pull their relevant *slaveHitIn* wire down. Arbiter would then be able to recognize which slave the master was addressing. Each master has a respective status register and an occupancy flag. The hit slave's ID is then stored in the relevant master's status register. The master's occupied flag will be set to 1. Then it goes to the ADDRESSED_STATE.

In the ADDRESSED_STATE, the arbiter checks if the hit slave is ready to perform the data phase transfer using the *swapGrantIn* signal. The slave would pull this high.

In some cases, the slave may be busy because the slave device is busy. The slave would then pull the *splitIn* signal high instead of the *swapGrantIn*. If the split in signal is high, the transaction would become a split transaction. The arbiter then pulls up another flag reserved for each master, the *mSplitOut* flag. Then The arbiter would go back to the IDLE state and service any other master's requests. The *mSplitOut* flag would block that master from completing any other operation. The occupancy flag would show the master is in the blocked state.

If the slave was not busy, and the data transaction is possible, the system would move back to the GRANT_STATE. Here the data phase part of the transaction would complete. Once the master completes the transaction, it pulls the *dismissedIn* signal high, indicating the transaction is over. This will move the arbiter to the IDLE state.

After a while, the slave device can become free to complete the transaction. The Slave would then pull the *unsplitIn* indicating the split transaction can complete. The *unsplitIn* has a higher priority than the other master's requests. (the reason for that is described in Special Case 01) . The arbiter then moves to the RELEASE_STATE.

In the release state it does multiple things. Sometimes there will be multiple masters in split state. Multiple slave devices can become free at the same time. Thus, it is important the higher priority master's split transaction is completed first. It then procedurally checks if each master's occupancy flag is high starting from the high priority master.

If the occupancy flag is high, it is an indication that master is in the split state. The arbiter would then check that master's status register to see which slave it was communicating with. It then checks if that slave generated an *unsplitIn* signal. If so, it sets *released_flag* and gives that master access to the bus via the GRANT_STATE. Here the data phase part of the transaction would complete. Once the master completes the transaction, it pulls the *dismissedIn* signal high, indicating the transaction is over.

If the occupancy flag is not high, it checks the next master.

Special Case 01

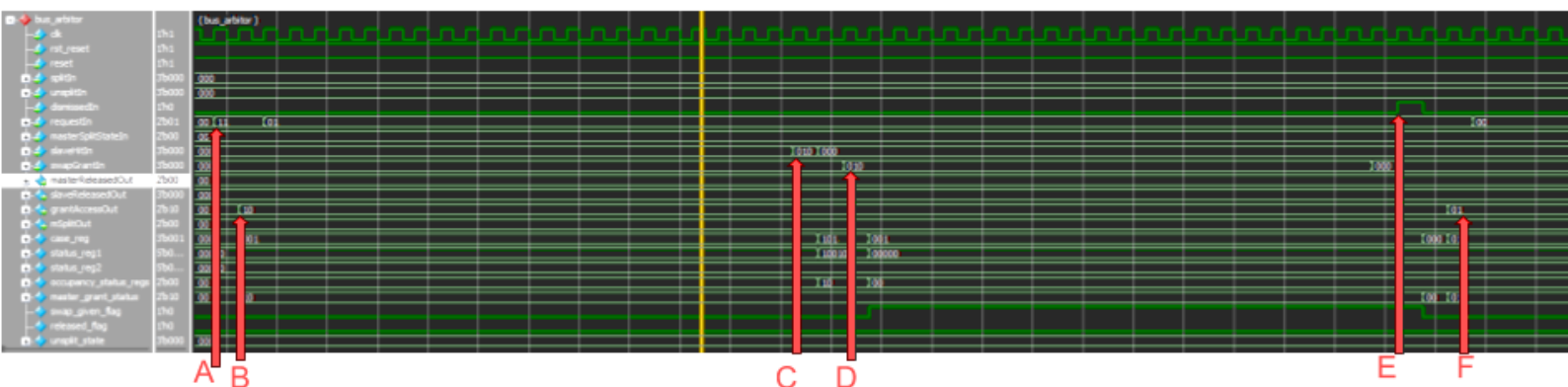
In some cases, after a transaction is split, a different master might have gotten access to the bus by the arbiter, but however it can attempt to access a slave which is already in the split state. However, this must be prevented. At this case, a dismissed signal will be initiated by the slave, it will remove the master's grant and the arbiter will go back to the idle state. However, this cycle can repeat multiple times until the slave becomes available by completing its split transaction. Now to make the slave available we want to make sure it completes its split transaction thus unsplit operations are given higher priority compared to master requests. (if not, a high priority master can repeatedly request split slave and block the split slave from becoming free by completing its split transaction, this will completely block the system)

IO Definition

Name	Source/Sink	Description
clk	Top	Clock
rst_reset	Top	This reset is to reset flops in reset synchronizer of the whole design.
reset	Top	Asynchronous reset of the whole design.
[NUM_OF_SLAVES - 1 : 0] splitIn	Slaves	A signal from the slave indicating the current transaction is being converted to a split transaction.
[NUM_OF_SLAVES - 1 : 0] unsplitIn	Slaves	A signal from the slave indicating that it is ready to complete and resume its split transaction as its slave device became available.
dismissedIn	Masters	This is a signal from the slave. If a master attempts to access a slave which is already in the split state, then this signal from the slave would remove the bus access from that master and bring the arbiter to an idle state.
[NUM_OF_MASTERS - 1 : 0] requestIn	Masters	Masters use this line to request the arbiter permission to use the BUS.
[NUM_OF_MASTERS - 1 : 0] masterSplitStateIn	Masters	Used to signal if which masters are in a split state.
[NUM_OF_MASTERS - 1 : 0] masterReleasedOut	Masters	Used to unblock a master from its split state so it can complete the rest of the split transaction.
[NUM_OF_MASTERS - 1 : 0] grantAccessOut	Masters	Arbiter uses this to give access to the bus for a specific bus. Additionally, it sets the multiplexors which direct the address phase and data phase data.
[NUM_OF_SLAVES - 1 : 0] slaveHitIn	Address Decoder	Indicates if the address is within the range of the slave
[NUM_OF_SLAVES - 1 : 0] swapGrantIn	Slaves	Indicates the slave is ready for data transfer.
[NUM_OF_SLAVES - 1 : 0] slaveReleasedOut	Slaves	Once the arbiter wants the slave to complete the split transaction. It uses this to signal out.
[NUM_OF_MASTERS - 1 : 0] mSplitOut	Masters	This is used to put a master into a blocked state.

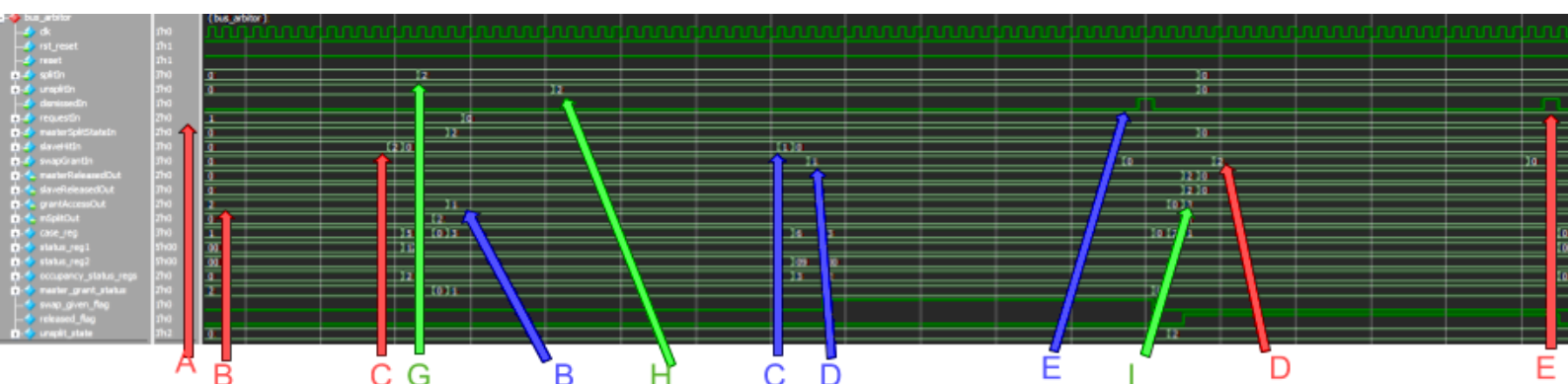
Timing Diagram

Normal Transaction



A: Here we see both the masters requesting, now at B, the arbiter gives access to the high priority master. Then the master accesses the address decoders. At C, you can notice one of the address decoders indicate that it is the one the master is referring. At D, the swapGrantIn is given out by the slave indicating it is ready for the data transfer. After this the data transfer occurs between the master and slave. At D, After the transaction is over, the master will use the dismissedIn signal to indicate the transaction is over.

Split Transaction



The A,B,C,D,E are the same as the previous.

The Red shows the initial transaction which was split. The Blue is another transaction. The Green signals are the signal relevant to the split operation. At G you can see the splitIn signal from the relevant slave. When the slave becomes available, it signals via the unsplitIn, which is the H signal. After the blue transaction is done, the arbiter goes back to handling the red transaction.

Master

Description and State Diagrams

A Master or Master Interface is something that interfaces a master device which is capable of initiating transactions. A master device can request the master to perform a particular operation such as read or write by giving it the address, data and the type of operation it wants done.

Then the master would continuously keep requesting the arbiter until it is given the grant signal. If the grant is given, it will send out the address and the type of operation to all the address decoders and wait for the arbiter's response. The address and type of operation is sent in the following format. {1} {01 if read and 10 if write} {Address}. This completes the first phase of the transaction, the address phase.

In some cases, the slave might initiate a split operation via the splitIn signal. Then the Master will be kept in a 'locked' state known as the split state by the arbiter. In this state, the master will not be able to do any other transaction until its split transaction is complete. It can become unblocked if the arbiter gives the releasedIn signal, then it can move to the data phase and complete the transaction. If the slave doesn't initiate the split operation, the transaction would move to the data phase.

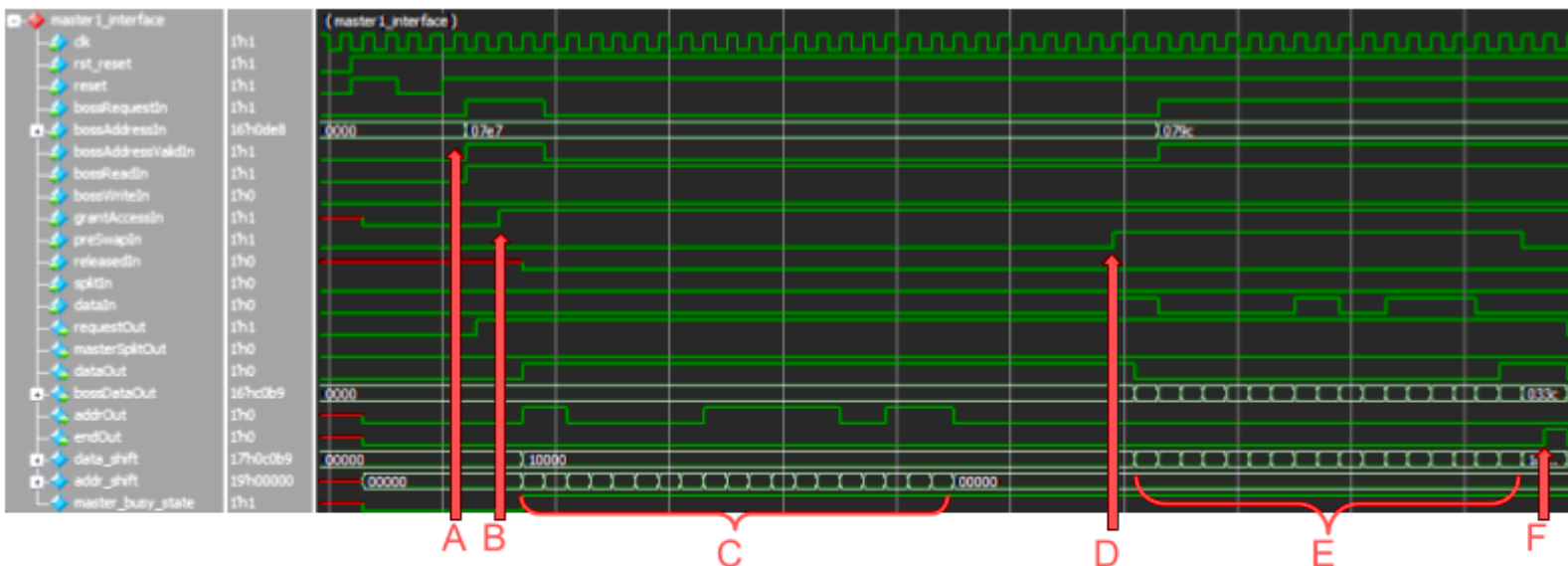
In the data phase, the moment the slave gives the preSwapIn signal, it simply swaps the data between the slave and its internal registers and presents it to the master device (if it is a read operation) and go back to an IDLE state. It signals the arbiter the transaction is complete using the dismissedOut signal.

IO Definition

Name	Source	Description
clk	Top	Clock
rst_reset	Top	This reset is to reset flops in reset synchronizer of the whole design.
reset	Top	Asynchronous reset of the whole design.
bossRequestIn	Master Device	This is the signal used by the master device to request for a transaction
[ADDR_BUS_WIDTH - 1 : 0] bossAddressIn	Master Device	The Master Device send the address bits parallelly into this port
bossAddressValidIn	Master Device	This indicates if the address in the BossAddressIn is valid or not
bossReadIn	Master Device	The Master device indicates if it wants a read operation done via this port on the given address.
bossWriteIn	Master Device	The Master device indicates if it wants a write operation done via this port on the given address for the given data via BOSS_DATA_IN
[DATA_BUS_WIDTH - 1 : 0] bossDataIn	Master Device	The Master Device writes whatever data it wants to be written to the slave here.
bossDataValidIn	Master Device	This indicates if the data in bossDataIn is valid or not.
splitIn	Arbiter	Signal from the slave indicating the transaction is being split
grantAccessIn	Bus Arbiter	The arbiter grants access to the BUS via this port. This signal initiates the transaction in the master interface.
preSwapIn	Slave	This is a signal by the slave indicating that it is ready to do the data transfer, or the data phase of the transaction.
releasedIn	Bus Arbiter	Once the arbiter wants the master to complete the split transaction, it will pull the releasedIn high, unblocking the master so it resumes its task.
dataIn	Slave	Data is taken in as a bitstream from the Slave
addrReadyIn	Address Decoder	This is a signal from the Address decoder if it is ready to accept an address phase transaction
dataAddrReadyOut	Master device	This is a signal from the Master indicating it is about to initiate a transaction and send out address and data.
masterSplitOut	Arbiter	If the master was blocked because it is doing a split transaction, it will continuously give this signal indicating it is blocked. Used by the arbiter to grant access to the bus once the split transaction resumes.
[DATA_BUS_WIDTH - 1 : 0] bossDataOut	Master Device	Data from the read operation is presented to the master device via this bus.
dataOut	Slave	Data is sent out via this port as a bitstream to the respective slave.
requestOut	Arbiter	The Master requests the arbiter for grant via this port
addrOut	addrDecoder	The serial port used to send the address bitstream
endOut	Arbiter and Master device	Master uses this to signal the end of a transaction and handover control of the bus to the arbiter
masterBusyState	Master Device	This indicates if the master is free to do a transaction to the Master Device.

Timing Diagrams

Read operation



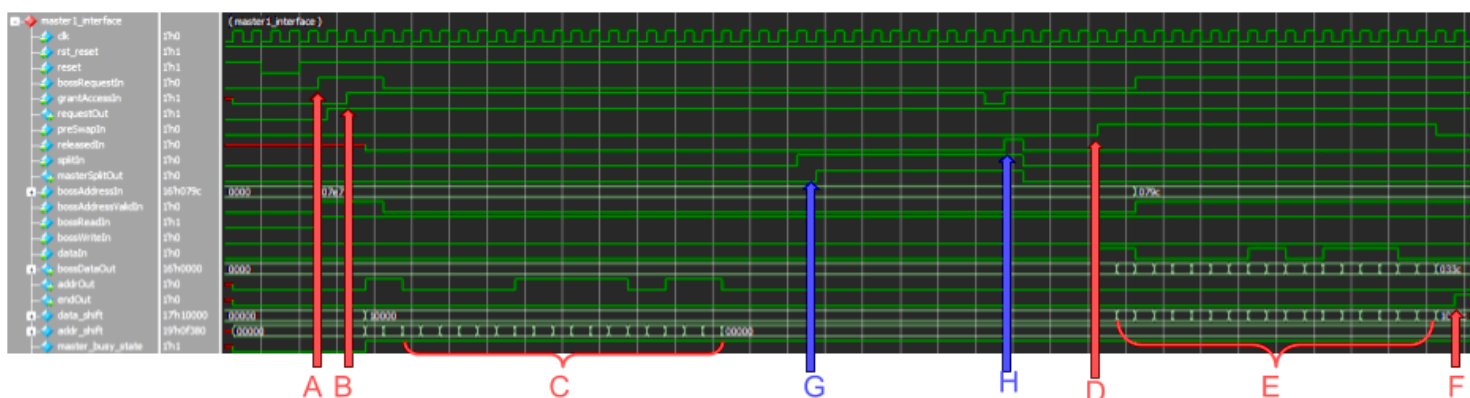
A: The Boss requests the master to do a transaction

B: Arbiter grants access to the bus for the master to do the transaction

C: Upon granting, it checks if the address decoder is ready for the transaction and sends out the address phase data serially to the address decoder

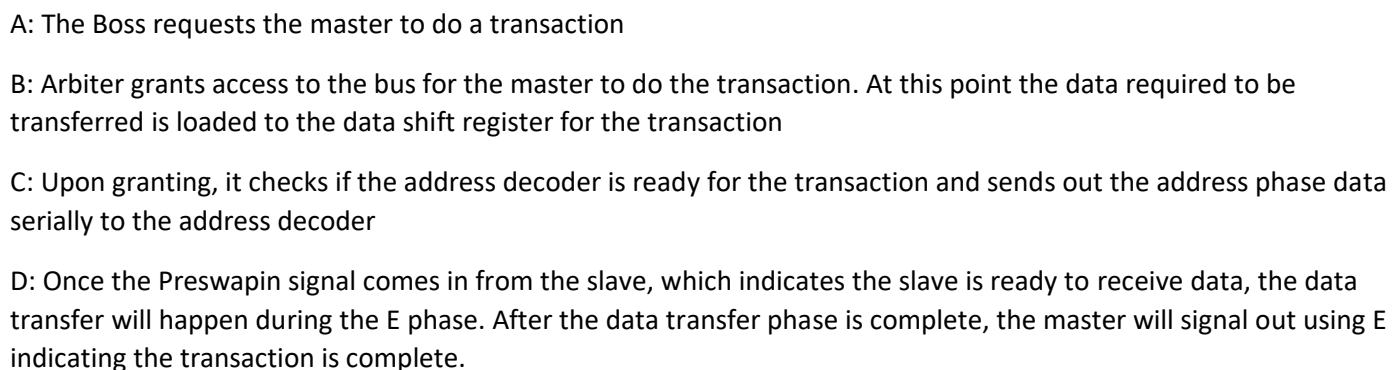
D: Once the Preswapin signal comes in from the slave, which indicates the slave is ready to send data, the data transfer will happen during the E phase. After the data transfer phase is complete, the master will signal out using E indicating the transaction is complete.

Read operation After Being Split in the Middle



Write Operation

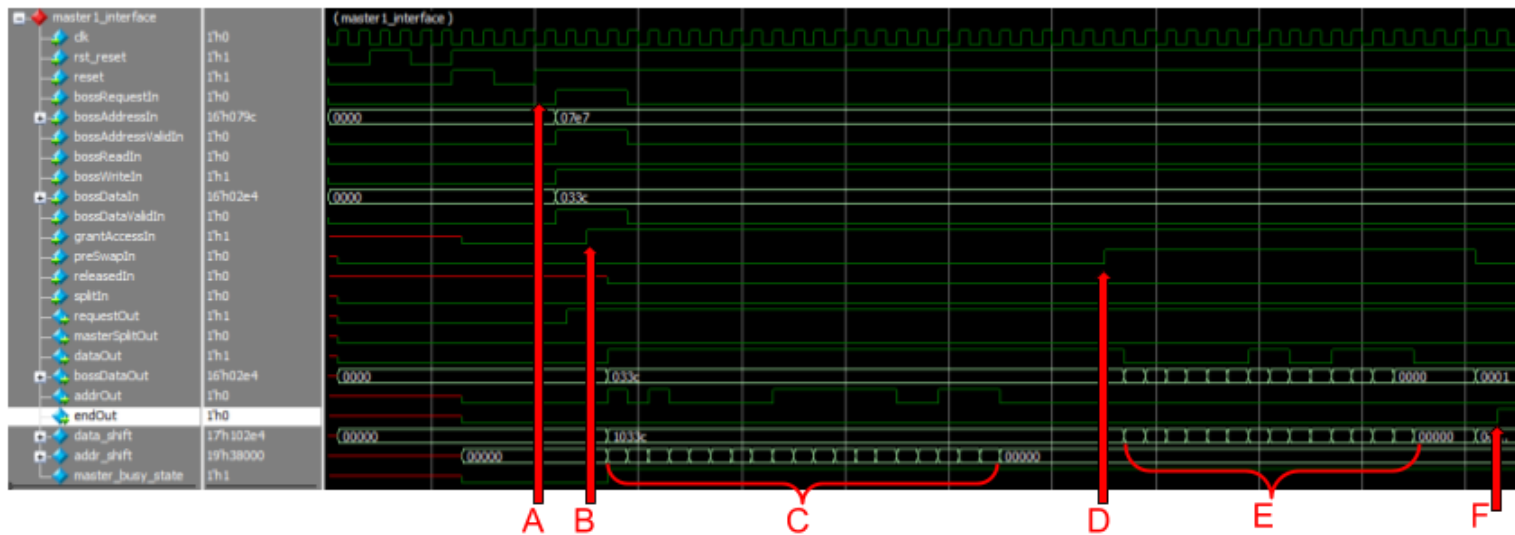
master_1_interface	(master_1_interface)
--------------------	----------------------



B: Arbiter grants access to the bus for the master to do the transaction. At this point the data required to be

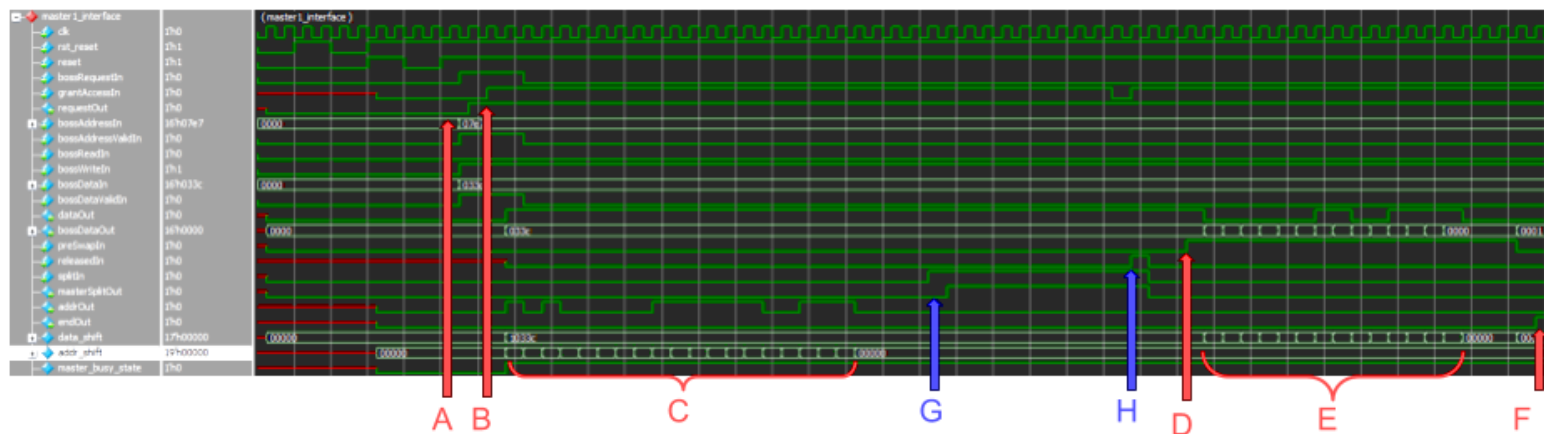
C: Upon granting, it checks if the address decoder is ready for the transaction and sends out the address phase data serially to the address decoder

D: Once the Preswapin signal comes in from the slave, which indicates the slave is ready to receive data, the data transfer will happen during the E phase. After the data transfer phase is complete, the master will signal out using E indicating the transaction is complete.



Write Operation After Being Split in the Middle

A,B,C,D,E,F are the same as the write operation tags. At G, after the receipt of the address, the slave checked to see if the slave device is available. However, since it was busy, it goes into the split state and inform the master and arbiter. Once the arbiter wants the master to resume and complete its split transaction, it will send out a released signal at H, which will let the master complete the remaining phase of the transaction.



Address Decoder

Description and State Diagrams

Each Slave interface or slave has an address decoder. This is a design choice to increase flexibility, code readability and scalability. It dramatically reduces the no. Of clock cycles taken for a transaction because the address is only serially transferred once. Thus, the address decoder is self-selecting.

Usually, all the slave's address decoders receive the address phase bitstream serially, which it loads to a shift register via the shift register's least significant bit register. The address phase bitstream's most significant bit is 1. The next two bits after the MSB in the address phase bitstream indicates if it is a read or write operation and the remaining bits the address. Once a complete address is loaded into the shift register, the shift register's MSB goes high (as per our protocol, the start bit of the address stream is always high), indicating a once empty shift register is now full.

Each address decoder would then check if the address portion of the data is within the corresponding slave device's range. If it is out of range, it will clean the shift register and ignore the signal.

If it is within the range, it will check if the slave is currently in a split state. If the slave is in a split state, it will inform that the transaction cannot be performed and inform the arbiter.

If the address is within the range and the slave is also not split and the slave is ready, it will send out an addrValidOut signal out to the slave indicating the next transaction phase can start. The address phase data would be given to the slave for the next part of the transaction.

IO Definition

Name	Source	Description
clk	Top	Clock Signal
rst_reset	Top	This reset is to reset flops in reset synchronizer of the whole design.
reset	Top	Asynchronous reset of the whole design.
addrIn	Master	Address phase data is received as a bitstream via the master here.
splitIn	Slaves	This signal comes from the Slave which indicates if the slave is currently in split state.
doneIn	Slaves	The address will be continuously available in the addrOut after a successful address transfer, once this signal comes the address decoder will wipe out its memory and go back to idle state.
addrReadyIn	Slaves	A Slave signal indicating if it is ready to accept address data.
[ADDR_OUT_BIT_WIDTH - 1] addrOut	Slave	The valid address phase data is given parallelly to the slave via this port.
addrValidOut	Slaves	This is a signal indicating that the address in AddrOut is a valid address.
ackSignalOut	Masters, Slave	This is a signal given out at the same time as addrValidOut which transfers the masters and slaves to the data transfer phase.
readyOut	Master	A General signal which indicates if the address decoder is idle. Once a complete address is loaded into the address decoder register, it goes low indicating it is not in that state.
slaveHitOut	Arbiter	This is a signal indicating the address send is within the slave's range

Timing Diagrams

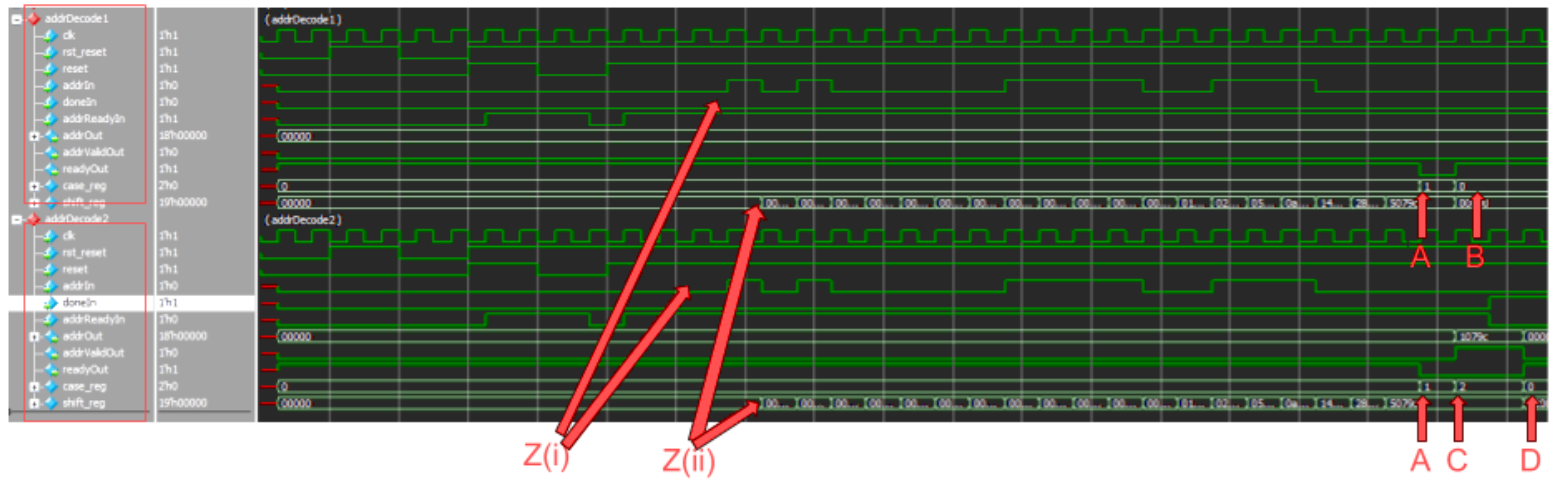
The following is a case where address data bitstream is being available to two address decoders. One of them is the target address decoder and the other is not. Z(i): Data is being send in via the "AddrIn" port of each address decoder. Simultaneously it is being loaded into each shift register of the address decoder at Z(ii).

A : At this point, the data has successfully been loaded to both the shift registers (MSB of the shift register becomes one) and it commences checking.

B: Here it was found out Address decoder 1 was not the target, thus it clears the register and goes back to idle state.

C: However, Address decoder 2 found it was the target, thus it loads the data and as the addrReadyIn was 1, it presents the data out via addrOut.

D: Because the doneIn signal was high, the address presented at AddrOut is cleared by clearing the shift register.



Slave Module

Description and State Diagrams

A Slave or Slave Interface is something that interfaces a Slave device.

After receiving the addrValidIn signal from the address decoder, the slave either initiates a Split transaction or completes the transaction depending on if the slave device is busy or not.

If the slave device is not busy. It will look at the address phase data to determine if it is a write operation or a read operation

READ Operation: First it checks if the read bit of the address phase bitstream is 1 or not. If so, it initiates a read transaction and goes into the FETCH_DATA_STATE. In this state, it requests the slave device for data for the address. Once the data is received, it will initiate a preSwapOut signal. This signal is used to synchronize the data transfer with the master. Then the data is sent out to the master and the operation is complete.

WRITE Operation: First it checks if the write bit of the address phase bitstream is 1 or not. If so, it initiates a write transaction by sending out a preSwapOut signal to the master. The data bitstream from the master is then loaded into its internal registers. Then the slave will send the data out to the slave device and get it written for that specific address. Once the write is successfully complete the slave ends the transaction.

If the slave device is not ready, the slave interface would initiate a split instruction. It goes to the SPLIT_STATE. It will remain in that state until the slave device is free and not busy. Once the slave device is available, it will go to the UNSPLIT_STATE and request the arbiter to complete the split transaction. The arbiter will release the slave from the split state using the releasedIn signal. Then the slave completes the remaining transaction.

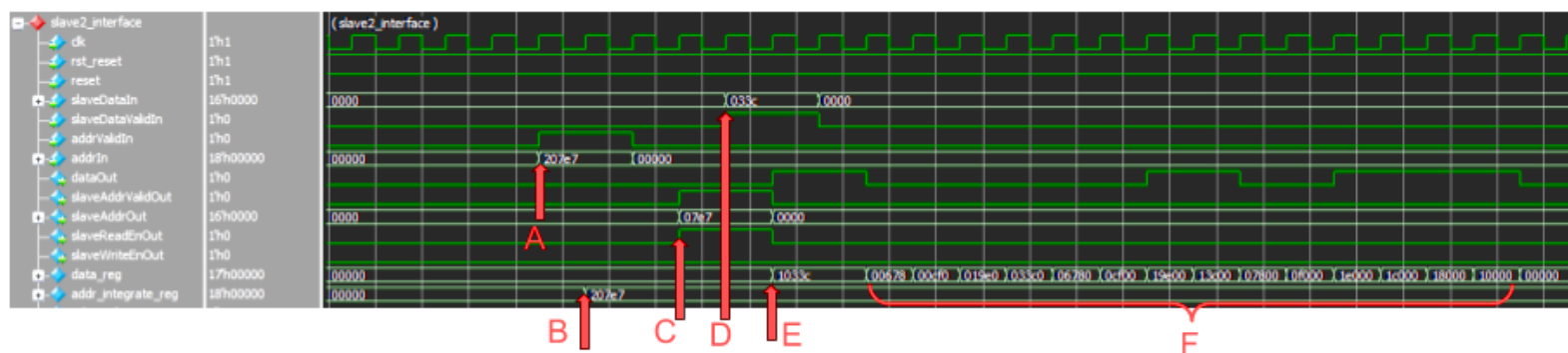
IO Definition

Name	Source	Description
clk	Clock	Clock
rst_reset	Out	This reset is to reset flops in reset synchronizer of the whole design.
reset	Out	Asynchronous reset of the whole design.
dataIn	Master	Data bitstream from the master comes here.
[DATA_BUS_WIDTH - 1 : 0] slaveDataIn	Slave Device	The data for the read operation is presented here by the slave device
slaveDataValidIn	Slave Device	This signal indicates the data for the read operation presented by the slaveDataIn is valid or not.
addrValidIn	Address Decoder	A signal indicating if the address data information in the addrIn is valid or not
[ADDR_IN_BIT_WIDTH - 1 : 0] addrIn	Address Decoder	Here the valid address phase data, such as the address, start bit, write and read bits are presented into the slave by the address decoder. All this information would be stored in a register for later use.
busyIn	Slave Device	This indicates the state of the slave device. If it is busy or available for the operation.
releasedIn	Bus Arbiter	
slaveWriteResponseIn	Slave Device	This is a signal indicating that the write operation done on the slave device with the address and data provided at slaveDataOut and slaveAddrOut is complete.
dataOut	Master	The Data bitstream to the slave is send from here.
slaveDataValidOut	Slave Device	This signal is pulled up if the data presented at slaveDataOut is stable
[DATA_BUS_WIDTH - 1 : 0] slaveDataOut	Slave Device	The data to be written to the Slave device it presented out from here.
slaveAddrValidOut	Slave Device	Indicates the address being presented in the slaveAddrOut is valid address for the slave device.
[ADDR_BUS_WIDTH - 1 : 0] slaveAddrOut	Slave Device	The address for read or write operation is presented to the slave device via this port.
splitOut	Arbiter	This indicates the Slave is currently in the Split state, meaning the slave device was busy
unsplitOut	Arbiter	Once the Slave device is not busy, the unsplit out signal brings the slave back to its UNSPLIT mode of operation, there it waits till the released signal from the bus arbiter. Once it receives the releasedIn from the bus arbiter, it can initiate the rest of the split transaction.

slaveReadEnOut	Slave Device	This sends a signal to the slave device indicating there is a read operation to be performed.
slaveWriteEnOut	Slave Device	This sends a signal to the slave device indicating there is a write operation to be performed.
addrReadyOut	Address Decoder	A signal from the slave indicating it is ready to receive addresses. Primarily send to the address decoder.
addrDoneOut	Address Decoder	An acknowledgement from the slave to the address decoder so the address decoder registers are cleared out.
preSwapOut	Master/Arbiter	Once the slave is ready the send out data to the master, it signals using the preSwapOut signal throughout the data transfer operation (predominantly used in the data phase of the transaction). This is used in both read and write operations.
swapGrantOut	Arbiter	This is a signal indicating that the slave is ready to move into the data transfer phase
slaveDataReadyOut	Slave device	Indicates the data is ready to be read in

Timing Diagrams

Read Operation



A: The Address phase data is presented here, and it is confirmed as valid via the `addrValidIn`.

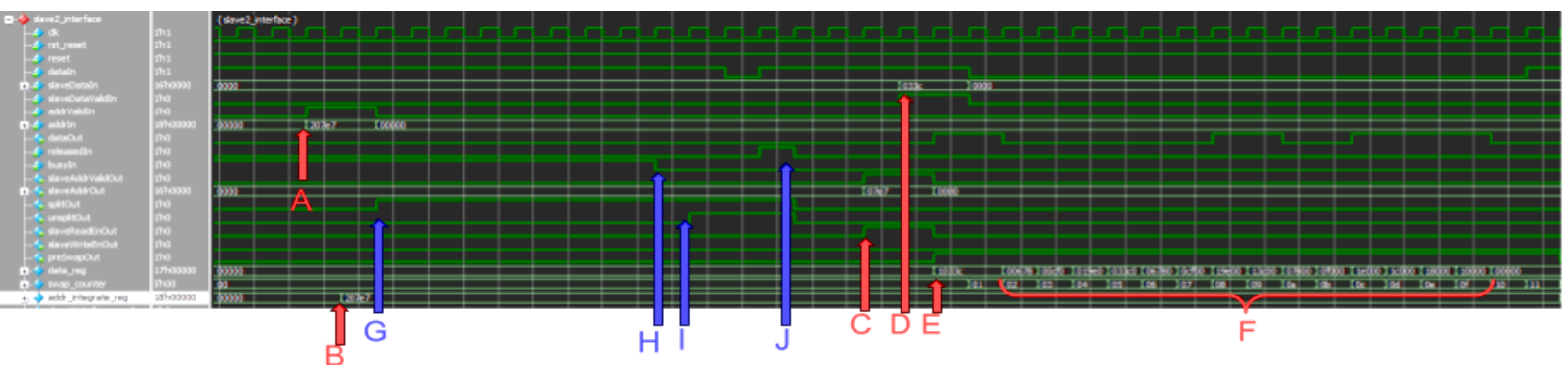
B: The Address phase data is loaded into the internal registers of the Slave here.

C: The Slave presents the address data to the slave device and requests for a read operation for that address.

D: Slave Device presents the data for the given address; the slave loads that data into its internal registers at E and prepares to send that data to the Master.

F: Here the data is sent out to the master, after which the slave goes into IDLE state.

Read Operation (after being Split in the middle)



The A,B,C,D,E,F are just as the same as the 'Read' operation tags. Here because the Slave Device was busy, at G, the slave goes into split mode of operation by pulling the splitOut high. It stays in this state until the slave device becomes available by pulling the busy to low. At this point the unsplitOut signal is send out to the bus arbiter requesting permission to use the bus to complete the transaction. Once bus arbiter sends out the released signal(J), the slave resumes the normal mode of operation.

Write Operation

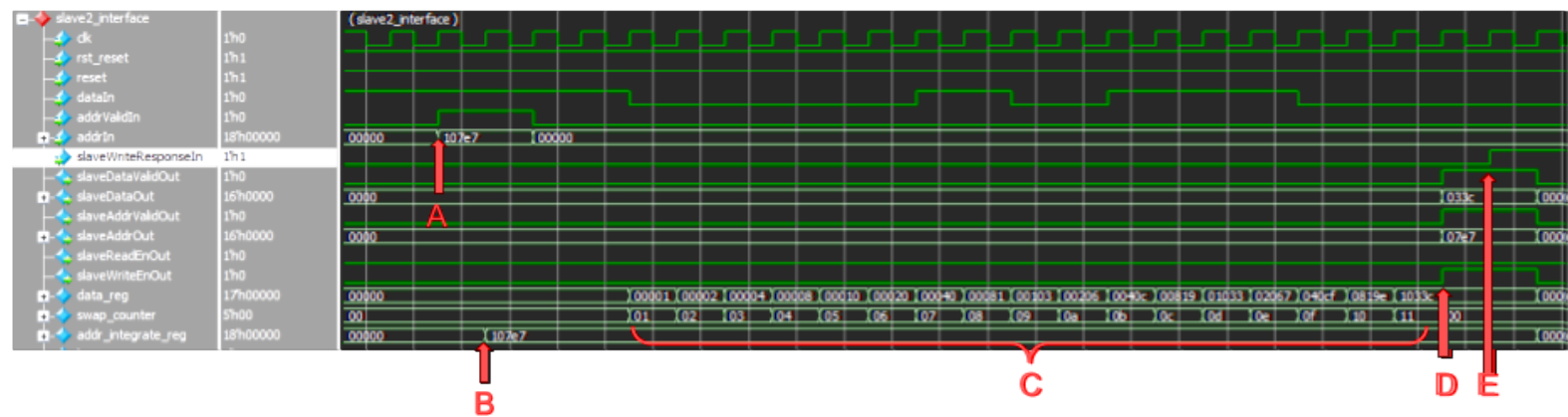
A: The Address phase data is presented here, and it is confirmed as valid via the addrValidIn.

B: The Address phase data is loaded into the internal registers of the Slave here.

C: The write data for the given address is loaded here from the master.

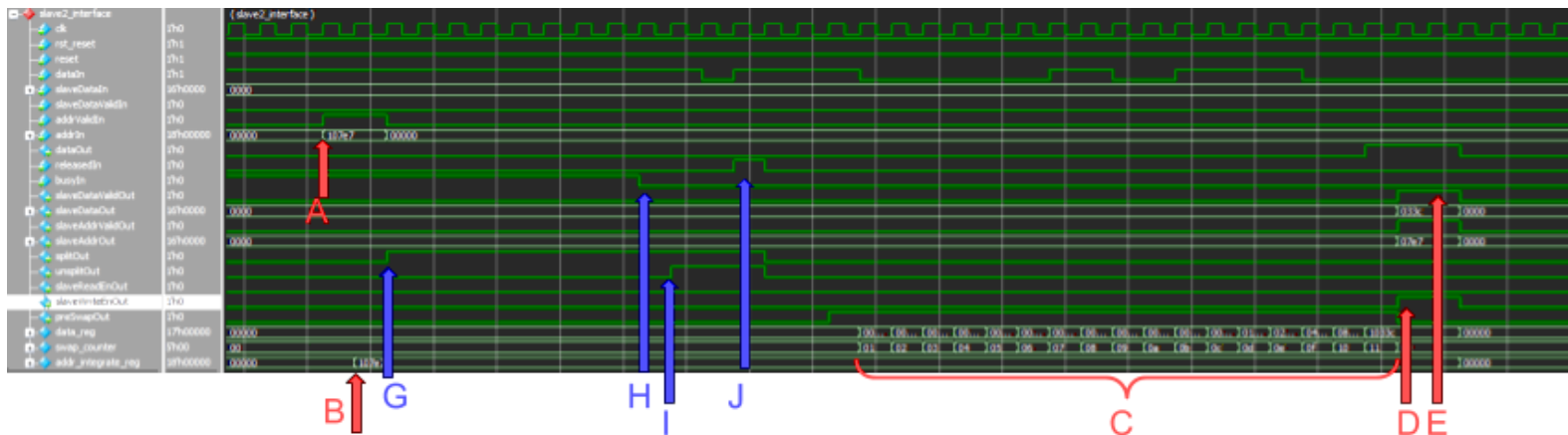
D: The address, and the data is then presented to the slave device for writing.

E: Once the slave device completes the write operation, it signals via the slaveWriteResponse indicating the transaction was successful.



Write Operation (after being Split in the middle)

The A,B,C,D,E,F are just as the same as the Write operation tags. Here because the Slave Device was busy, at G, the slave goes into split mode of operation by pulling the splitOut high. It stays in this state until the slave device becomes available by pulling the busy to low. At this point the unsplitOut signal is send out to the bus arbiter requesting permission to use the bus to complete the transaction. Once bus arbiter sends out the released signal(J), the slave resumes the normal mode of operation.



RTL

1.TOP

2.BUS ARBITER

3.ADDRESS DECODER

4.MASTER

5.SLAVE

```

1  `timescale 1ns / 1ps
2  //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
3  // Company: ENTC, University of Moratuwa
4  // Author: Miyuru Thathsara | Nadarasar Bahavan
5  //
6  // Create Date: 12/17/2020 02:15:54 AM
7  // Design Name: Top module
8  // Module Name: top
9  // Project Name: System Bus
10 // Additional Comments:
11 //
12 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
13 module top(
14     clk,
15     rst_reset,
16     reset,
17
18     boss1RequestIn,
19     boss2RequestIn,
20
21     boss1AddressIn,
22     boss2AddressIn,
23
24     boss1AddressValidIn,
25     boss2AddressValidIn,
26
27     boss1ReadIn,
28     boss2ReadIn,
29
30     boss1WriteIn,
31     boss2WriteIn,
32
33     boss1DataIn,
34     boss2DataIn,
35
36     boss1DataValidIn,
37     boss2DataValidIn,
38
39     slavel1DataIn,
40     slave2DataIn,
41     slave3DataIn,
42
43     slavel1DataValidIn,
44     slave2DataValidIn,
45     slave3DataValidIn,
46
47     slavel1WriteResponseIn,
48     slave2WriteResponseIn,
49     slave3WriteResponseIn,
50
51     slavel1BusyIn,
52     slave2BusyIn,
53     slave3BusyIn,
54
55     boss1DataOut,
56     boss2DataOut,
57
58     boss1DataValidOut,
59     boss2DataValidOut,
60
61     slavel1DataValidOut,
62     slave2DataValidOut,
63     slave3DataValidOut,
64
65     slavel1DataOut,
66     slave2DataOut,
67     slave3DataOut,
68
69     slavel1AddrValidOut,

```

```

70     slave2AddrValidOut,
71     slave3AddrValidOut,
72
73     slave1AddrOut,
74     slave2AddrOut,
75     slave3AddrOut,
76
77     slave1ReadEnOut,
78     slave2ReadEnOut,
79     slave3ReadEnOut,
80
81     slave1WriteEnOut,
82     slave2WriteEnOut,
83     slave3WriteEnOut,
84
85     slave1DataReadyOut,
86     slave2DataReadyOut,
87     slave3DataReadyOut,
88
89     boss1DataAddrReadyOut,
90     boss2DataAddrReadyOut,
91
92     boss1NoAddressOut,
93     boss2NoAddressOut,
94
95     boss1TransIncompleteOut,
96     boss2TransIncompleteOut
97 );
98
99 // Parameters
100
101 parameter MASTER_INT_STATE_REG_WIDTH           = 3;
102 parameter ADDR_DECODE_STATE_REG_WIDTH          = 2;
103 parameter NUM_OF_MASTERS                        = 2;
104 parameter NUM_OF_SLAVES                        = 3;
105 parameter DATA_BUS_WIDTH                      = 16;
106 parameter ADDR_BUS_WIDTH                      = 16;
107 parameter ADDR_SERIAL_BIT_WIDTH                = 19;
108 parameter DATA_SERIAL_BIT_WIDTH               = 17;
109 parameter SLAVE1_ADDR_RANGE_BEGIN              = 0;
110 parameter SLAVE1_ADDR_RANGE_END                = 1024;
111 parameter SLAVE2_ADDR_RANGE_BEGIN              = 1024;
112 parameter SLAVE2_ADDR_RANGE_END                = 2048;
113 parameter SLAVE3_ADDR_RANGE_BEGIN              = 2048;
114 parameter SLAVE3_ADDR_RANGE_END                = 4096;
115
116 // Local Parameters
117
118 localparam ADDR_OUT_BIT_WIDTH                  =
119 ADDR_SERIAL_BIT_WIDTH - 1;
120
121 // I/O Pins
122
123 input                                     clk;
124 input                                     rst_reset;
125 input                                     reset;
126 input                                     boss1RequestIn;
127 input                                     boss2RequestIn;
128 input                                     boss1AddressIn;
129 input                                     boss2AddressIn;
130
131 input                                     boss1AddressValidIn;
132
133 input                                     boss2AddressValidIn;
134
135 input                                     boss1ReadIn;
136 input                                     boss2ReadIn;
137 input                                     boss1WriteIn;
138 input                                     boss2WriteIn;
139 input                                     boss1DataIn;
140 input                                     boss2DataIn;
141
142 // Slave 1
143
144 input                                     slave1AddrValidIn;
145 input                                     slave1DataValidIn;
146 input                                     slave1WriteEnIn;
147 input                                     slave1ReadEnIn;
148 input                                     slave1DataReadyIn;
149
150 input                                     slave1DataIn;
151 input                                     slave1WriteDataIn;
152 input                                     slave1ReadDataIn;
153
154 // Slave 2
155
156 input                                     slave2AddrValidIn;
157 input                                     slave2DataValidIn;
158 input                                     slave2WriteEnIn;
159 input                                     slave2ReadEnIn;
160 input                                     slave2DataReadyIn;
161
162 input                                     slave2DataIn;
163 input                                     slave2WriteDataIn;
164 input                                     slave2ReadDataIn;
165
166 // Slave 3
167
168 input                                     slave3AddrValidIn;
169 input                                     slave3DataValidIn;
170 input                                     slave3WriteEnIn;
171 input                                     slave3ReadEnIn;
172 input                                     slave3DataReadyIn;
173
174 input                                     slave3DataIn;
175 input                                     slave3WriteDataIn;
176 input                                     slave3ReadDataIn;
177
178 // Boss 1
179
180 input                                     boss1DataValidIn;
181 input                                     boss1DataReadyIn;
182 input                                     boss1TransCompleteIn;
183
184 input                                     boss1DataIn;
185 input                                     boss1WriteDataIn;
186 input                                     boss1ReadDataIn;
187
188 // Boss 2
189
190 input                                     boss2DataValidIn;
191 input                                     boss2DataReadyIn;
192 input                                     boss2TransCompleteIn;
193
194 input                                     boss2DataIn;
195 input                                     boss2WriteDataIn;
196 input                                     boss2ReadDataIn;
197
198 // Master
199
200 input                                     masterDataValidIn;
201 input                                     masterDataReadyIn;
202 input                                     masterTransCompleteIn;
203
204 input                                     masterDataIn;
205 input                                     masterWriteDataIn;
206 input                                     masterReadDataIn;
207
208 // Internal
209
210 input                                     intStateReg;
211 input                                     decodeStateReg;
212 input                                     masterReq;
213 input                                     slave1Req;
214 input                                     slave2Req;
215 input                                     slave3Req;
216 input                                     slave1Data;
217 input                                     slave2Data;
218 input                                     slave3Data;
219 input                                     slave1WriteData;
220 input                                     slave2WriteData;
221 input                                     slave3WriteData;
222 input                                     slave1ReadData;
223 input                                     slave2ReadData;
224 input                                     slave3ReadData;
225 input                                     boss1Data;
226 input                                     boss2Data;
227 input                                     boss1WriteData;
228 input                                     boss2WriteData;
229 input                                     boss1ReadData;
230 input                                     boss2ReadData;
231
232 // Internal
233
234 input                                     intStateReg;
235 input                                     decodeStateReg;
236 input                                     masterReq;
237 input                                     slave1Req;
238 input                                     slave2Req;
239 input                                     slave3Req;
240 input                                     slave1Data;
241 input                                     slave2Data;
242 input                                     slave3Data;
243 input                                     slave1WriteData;
244 input                                     slave2WriteData;
245 input                                     slave3WriteData;
246 input                                     slave1ReadData;
247 input                                     slave2ReadData;
248 input                                     slave3ReadData;
249 input                                     boss1Data;
250 input                                     boss2Data;
251 input                                     boss1WriteData;
252 input                                     boss2WriteData;
253 input                                     boss1ReadData;
254 input                                     boss2ReadData;
255
256 // Internal
257
258 input                                     intStateReg;
259 input                                     decodeStateReg;
260 input                                     masterReq;
261 input                                     slave1Req;
262 input                                     slave2Req;
263 input                                     slave3Req;
264 input                                     slave1Data;
265 input                                     slave2Data;
266 input                                     slave3Data;
267 input                                     slave1WriteData;
268 input                                     slave2WriteData;
269 input                                     slave3WriteData;
270 input                                     slave1ReadData;
271 input                                     slave2ReadData;
272 input                                     slave3ReadData;
273 input                                     boss1Data;
274 input                                     boss2Data;
275 input                                     boss1WriteData;
276 input                                     boss2WriteData;
277 input                                     boss1ReadData;
278 input                                     boss2ReadData;
279
280 // Internal
281
282 input                                     intStateReg;
283 input                                     decodeStateReg;
284 input                                     masterReq;
285 input                                     slave1Req;
286 input                                     slave2Req;
287 input                                     slave3Req;
288 input                                     slave1Data;
289 input                                     slave2Data;
290 input                                     slave3Data;
291 input                                     slave1WriteData;
292 input                                     slave2WriteData;
293 input                                     slave3WriteData;
294 input                                     slave1ReadData;
295 input                                     slave2ReadData;
296 input                                     slave3ReadData;
297 input                                     boss1Data;
298 input                                     boss2Data;
299 input                                     boss1WriteData;
300 input                                     boss2WriteData;
301 input                                     boss1ReadData;
302 input                                     boss2ReadData;
303
304 // Internal
305
306 input                                     intStateReg;
307 input                                     decodeStateReg;
308 input                                     masterReq;
309 input                                     slave1Req;
310 input                                     slave2Req;
311 input                                     slave3Req;
312 input                                     slave1Data;
313 input                                     slave2Data;
314 input                                     slave3Data;
315 input                                     slave1WriteData;
316 input                                     slave2WriteData;
317 input                                     slave3WriteData;
318 input                                     slave1ReadData;
319 input                                     slave2ReadData;
320 input                                     slave3ReadData;
321 input                                     boss1Data;
322 input                                     boss2Data;
323 input                                     boss1WriteData;
324 input                                     boss2WriteData;
325 input                                     boss1ReadData;
326 input                                     boss2ReadData;
327
328 // Internal
329
330 input                                     intStateReg;
331 input                                     decodeStateReg;
332 input                                     masterReq;
333 input                                     slave1Req;
334 input                                     slave2Req;
335 input                                     slave3Req;
336 input                                     slave1Data;
337 input                                     slave2Data;
338 input                                     slave3Data;
339 input                                     slave1WriteData;
340 input                                     slave2WriteData;
341 input                                     slave3WriteData;
342 input                                     slave1ReadData;
343 input                                     slave2ReadData;
344 input                                     slave3ReadData;
345 input                                     boss1Data;
346 input                                     boss2Data;
347 input                                     boss1WriteData;
348 input                                     boss2WriteData;
349 input                                     boss1ReadData;
350 input                                     boss2ReadData;
351
352 // Internal
353
354 input                                     intStateReg;
355 input                                     decodeStateReg;
356 input                                     masterReq;
357 input                                     slave1Req;
358 input                                     slave2Req;
359 input                                     slave3Req;
360 input                                     slave1Data;
361 input                                     slave2Data;
362 input                                     slave3Data;
363 input                                     slave1WriteData;
364 input                                     slave2WriteData;
365 input                                     slave3WriteData;
366 input                                     slave1ReadData;
367 input                                     slave2ReadData;
368 input                                     slave3ReadData;
369 input                                     boss1Data;
370 input                                     boss2Data;
371 input                                     boss1WriteData;
372 input                                     boss2WriteData;
373 input                                     boss1ReadData;
374 input                                     boss2ReadData;
375
376 // Internal
377
378 input                                     intStateReg;
379 input                                     decodeStateReg;
380 input                                     masterReq;
381 input                                     slave1Req;
382 input                                     slave2Req;
383 input                                     slave3Req;
384 input                                     slave1Data;
385 input                                     slave2Data;
386 input                                     slave3Data;
387 input                                     slave1WriteData;
388 input                                     slave2WriteData;
389 input                                     slave3WriteData;
390 input                                     slave1ReadData;
391 input                                     slave2ReadData;
392 input                                     slave3ReadData;
393 input                                     boss1Data;
394 input                                     boss2Data;
395 input                                     boss1WriteData;
396 input                                     boss2WriteData;
397 input                                     boss1ReadData;
398 input                                     boss2ReadData;
399
400 // Internal
401
402 input                                     intStateReg;
403 input                                     decodeStateReg;
404 input                                     masterReq;
405 input                                     slave1Req;
406 input                                     slave2Req;
407 input                                     slave3Req;
408 input                                     slave1Data;
409 input                                     slave2Data;
410 input                                     slave3Data;
411 input                                     slave1WriteData;
412 input                                     slave2WriteData;
413 input                                     slave3WriteData;
414 input                                     slave1ReadData;
415 input                                     slave2ReadData;
416 input                                     slave3ReadData;
417 input                                     boss1Data;
418 input                                     boss2Data;
419 input                                     boss1WriteData;
420 input                                     boss2WriteData;
421 input                                     boss1ReadData;
422 input                                     boss2ReadData;
423
424 // Internal
425
426 input                                     intStateReg;
427 input                                     decodeStateReg;
428 input                                     masterReq;
429 input                                     slave1Req;
430 input                                     slave2Req;
431 input                                     slave3Req;
432 input                                     slave1Data;
433 input                                     slave2Data;
434 input                                     slave3Data;
435 input                                     slave1WriteData;
436 input                                     slave2WriteData;
437 input                                     slave3WriteData;
438 input                                     slave1ReadData;
439 input                                     slave2ReadData;
440 input                                     slave3ReadData;
441 input                                     boss1Data;
442 input                                     boss2Data;
443 input                                     boss1WriteData;
444 input                                     boss2WriteData;
445 input                                     boss1ReadData;
446 input                                     boss2ReadData;
447
448 // Internal
449
450 input                                     intStateReg;
451 input                                     decodeStateReg;
452 input                                     masterReq;
453 input                                     slave1Req;
454 input                                     slave2Req;
455 input                                     slave3Req;
456 input                                     slave1Data;
457 input                                     slave2Data;
458 input                                     slave3Data;
459 input                                     slave1WriteData;
460 input                                     slave2WriteData;
461 input                                     slave3WriteData;
462 input                                     slave1ReadData;
463 input                                     slave2ReadData;
464 input                                     slave3ReadData;
465 input                                     boss1Data;
466 input                                     boss2Data;
467 input                                     boss1WriteData;
468 input                                     boss2WriteData;
469 input                                     boss1ReadData;
470 input                                     boss2ReadData;
471
472 // Internal
473
474 input                                     intStateReg;
475 input                                     decodeStateReg;
476 input                                     masterReq;
477 input                                     slave1Req;
478 input                                     slave2Req;
479 input                                     slave3Req;
480 input                                     slave1Data;
481 input                                     slave2Data;
482 input                                     slave3Data;
483 input                                     slave1WriteData;
484 input                                     slave2WriteData;
485 input                                     slave3WriteData;
486 input                                     slave1ReadData;
487 input                                     slave2ReadData;
488 input                                     slave3ReadData;
489 input                                     boss1Data;
490 input                                     boss2Data;
491 input                                     boss1WriteData;
492 input                                     boss2WriteData;
493 input                                     boss1ReadData;
494 input                                     boss2
```

136	input	[DATA_BUS_WIDTH - 1 : 0]	boss2DataIn;
137	input		boss1DataValidIn;
138	input		boss2DataValidIn;
139	input	[DATA_BUS_WIDTH - 1 : 0]	slave1DataIn;
140	input	[DATA_BUS_WIDTH - 1 : 0]	slave2DataIn;
141	input	[DATA_BUS_WIDTH - 1 : 0]	slave3DataIn;
142	input		
		slave1DataValidIn;	
143	input		
		slave2DataValidIn;	
144	input		
		slave3DataValidIn;	
145	input		
		slave1WriteResponseIn;	
146	input		
		slave2WriteResponseIn;	
147	input		
		slave3WriteResponseIn;	
148	input		slave1BusyIn;
149	input		slave2BusyIn;
150	input		slave3BusyIn;
151	output	[DATA_BUS_WIDTH - 1 : 0]	boss1DataOut;
152	output	[DATA_BUS_WIDTH - 1 : 0]	boss2DataOut;
153	output		
		boss1DataValidOut;	
154	output		
		boss2DataValidOut;	
155	output		
		slave1DataValidOut;	
156	output		
		slave2DataValidOut;	
157	output		
		slave3DataValidOut;	
158	output	[DATA_BUS_WIDTH - 1 : 0]	slave1DataOut;
159	output	[DATA_BUS_WIDTH - 1 : 0]	slave2DataOut;
160	output	[DATA_BUS_WIDTH - 1 : 0]	slave3DataOut;
161	output		
		slave1AddrValidOut;	
162	output		
		slave2AddrValidOut;	
163	output		
		slave3AddrValidOut;	
164	output	[ADDR_BUS_WIDTH - 1 : 0]	slave1AddrOut;
165	output	[ADDR_BUS_WIDTH - 1 : 0]	slave2AddrOut;
166	output	[ADDR_BUS_WIDTH - 1 : 0]	slave3AddrOut;
167	output		slave1ReadEnOut;
168	output		slave2ReadEnOut;
169	output		slave3ReadEnOut;
170	output		slave1WriteEnOut;
171	output		slave2WriteEnOut;
172	output		slave3WriteEnOut;
173	output		
		slave1DataReadyOut;	
174	output		
		slave2DataReadyOut;	
175	output		
		slave3DataReadyOut;	
176	output		
		boss1DataAddrReadyOut;	
177	output		
		boss2DataAddrReadyOut;	
178	output		
		boss1NoAddressOut;	
179	output		
		boss2NoAddressOut;	
180	output		
		boss1TransIncompleteOut;	
181	output		
		boss2TransIncompleteOut;	

```

182 ////////////////////////////////////////////////////
183 // Wires and Registers
184 ////////////////////////////////////////////////////
185 wire master1_request;
186 wire master2_request;
187
188 master1_split_state;
189 wire master2_split_state;
190 wire master1_grant_access;
191 wire master2_grant_access;
192
193 wire slave1_split;
194 wire slave2_split;
195 wire slave3_split;
196 wire slave1_unsplit;
197 wire slave2_unsplit;
198 wire slave3_unsplit;
199 reg master1_data_in;
200 reg master2_data_in;
201 wire master1_data_out;
202 wire master2_data_out;
203 wire slave1_data_out;
204 wire slave2_data_out;
205 wire slave3_data_out;
206
207 slave1_addr_ready;
208 wire slave2_addr_ready;
209 wire slave3_addr_ready;
210
211 wire slave1_preswap;
212 wire slave2_preswap;
213 wire slave3_preswap;
214 wire boss1_released;
215 wire boss2_released;
216
217 slave1_addr_valid;
218 wire slave2_addr_valid;
219 wire slave3_addr_valid;
220
221 wire [ ADDR_OUT_BIT_WIDTH - 1 : 0 ] slave1_addr;
222 wire [ ADDR_OUT_BIT_WIDTH - 1 : 0 ] slave2_addr;
223 wire [ ADDR_OUT_BIT_WIDTH - 1 : 0 ] slave3_addr;
224 wire slave1_done;
225 wire slave2_done;
226 wire slave3_done;
227
228 addrdecode1_ready;
229 wire addrdecode2_ready;
230 wire addrdecode3_ready;
231
232 wire master1_addr_out;
233 wire master2_addr_out;
234 wire master1_split;
235 wire master2_split;
236 wire slave1_hit;
237 wire slave2_hit;
238 wire slave3_hit;
239
240 slave1_swap_grant;
241 wire slave2_swap_grant;
242 wire slave3_swap_grant;
243
244 wire slave1_released;

```

```

235 wire slave2_released;
236 wire slave3_released;
237 wire
master1_no_address;
238 wire
master2_no_address;
239 wire slave1_not_hit;
240 wire slave2_not_hit;
241 wire slave3_not_hit;
242 wire
master1_arbitor_slave_split;
243 wire
master2_arbitor_slave_split;
244 wire [ NUM_OF_MASTERS - 1 : 0 ]
master_arbitor_slave_split_wire;
245 wire [ NUM_OF_SLAVES - 1 : 0 ] swap_grant_wire;
246 wire [ NUM_OF_SLAVES - 1 : 0 ] slave_hit_wire;
247 wire [ NUM_OF_MASTERS - 1 : 0 ] m_split_wire;
248 reg
master2slave_addr;
249 wire preswap_wire;
250 wire
addrdecode_ready_wire;
251 wire [ NUM_OF_SLAVES - 1 : 0 ] split_wire;
252 wire [ NUM_OF_SLAVES - 1 : 0 ] unsplit_wire;
253 wire dismissed_wire;
254 wire [ NUM_OF_MASTERS - 1 : 0 ]
master_released_wire;
255 wire [ NUM_OF_SLAVES - 1 : 0 ]
slave_released_wire;
256 wire [ NUM_OF_MASTERS - 1 : 0 ] request_wire;
257 wire [ NUM_OF_MASTERS - 1 : 0 ]
master_split_state_wire;
258 wire [ NUM_OF_MASTERS - 1 : 0 ]
grant_access_wire;
259 reg
master2slave_data;
260 wire
slave2master_data;
261 wire [ NUM_OF_MASTERS - 1 : 0 ]
master_no_address;
262 wire [ NUM_OF_SLAVES - 1 : 0 ] slave_not_hit;
263 //////////////////////////////////////////////////
264 // Implementation
265 //////////////////////////////////////////////////
266
267 assign request_wire = {
master1_request, master2_request };
268 assign master_split_state_wire = {
master1_split_state, master2_split_state };
269 assign master1_grant_access =
grant_access_wire[1];
270 assign master2_grant_access =
grant_access_wire[0];
271 assign split_wire = {
slave1_split, slave2_split, slave3_split };
272 assign master1_split =
m_split_wire[1];
273 assign master2_split =
m_split_wire[0];
274 assign unsplit_wire = {
slave1_unsplit, slave2_unsplit, slave3_unsplit };
275 assign dismissed_wire =
boss1DataValidOut | boss2DataValidOut;
276 assign slave2master_data =
slave1_data_out | slave2_data_out | slave3_data_out;
277 assign addrdecode_ready_wire =
addrdecode1_ready | addrdecode2_ready | addrdecode3_ready;
278 assign preswap_wire =

```

```

279 slave1_preswap | slave2_preswap | slave3_preswap;
assign boss1_released                                     =
master_released_wire[1];
280 assign boss2_released                                     =
master_released_wire[0];
281 assign slave1_released                                     =
slave_released_wire[2];
282 assign slave2_released                                     =
slave_released_wire[1];
283 assign slave3_released                                     =
slave_released_wire[0];
284 assign slave_hit_wire                                     = { slave1_hit,
slave2_hit, slave3_hit };
285 assign swap_grant_wire                                     = {
slave1_swap_grant, slave2_swap_grant, slave3_swap_grant };
286 assign master1_no_address                                     =
master_no_address[1];
287 assign master2_no_address                                     =
master_no_address[0];
288 assign slave_not_hit                                     = {
slave1_not_hit, slave2_not_hit, slave3_not_hit };
289 assign master1_arbitor_slave_split                                     =
master_arbitor_slave_split_wire[1];
290 assign master2_arbitor_slave_split                                     =
master_arbitor_slave_split_wire[0];
291
292 always@(*)begin
293     if(grant_access_wire[1] == 1'b1)begin
294         master2slave_data                                     =
master1_data_out;
295         master2slave_addr                                     =
master1_addr_out;
296         master1_data_in                                     =
slave2master_data;
297     end
298     else begin
299         if(grant_access_wire[0] == 1'b1)begin
300             master2slave_data                                     =
master2_data_out;
301             master2slave_addr                                     =
master2_addr_out;
302             master2_data_in                                     =
slave2master_data;
303         end
304         else begin
305             master1_data_in                                     = 1'b0;
306             master2_data_in                                     = 1'b0;
307             master2slave_data                                     = 1'b0;
308             master2slave_addr                                     = 1'b0;
309         end
310     end
311 end
312
313 busArbitor#(
314     .NUM_OF_MASTERS(NUM_OF_MASTERS),
315     .NUM_OF_SLAVES(NUM_OF_SLAVES)
316 )
317 busArbitor(
318     .clk(clk),
319     .rst_reset(rst_reset),
320     .reset(reset),
321     .requestIn(request_wire),
322     .splitIn(split_wire),
323     .unsplitIn(unsplit_wire),
324     .dismissedIn(dismissed_wire),
325     .masterReleasedOut(master_released_wire),
326     .slaveReleasedOut(slave_released_wire),
327     .masterSplitStateIn(master_split_state_wire),
328     .slaveHitIn(slave_hit_wire),

```

```

329     .slaveNotHitIn(slave_not_hit),
330     .swapGrantIn(swap_grant_wire),
331     .grantAccessOut(grant_access_wire),
332     .mSplitOut(m_split_wire),
333     .noAddressOut(master_no_address),
334     .slaveSplitOut(master_arbitor_slave_split_wire)
335 );
336
337 masterInterface#(
338     .STATE_REG_WIDTH(MASTER_INT_STATE_REG_WIDTH),
339     .DATA_BUS_WIDTH(DATA_BUS_WIDTH),
340     .ADDR_BUS_WIDTH(ADDR_BUS_WIDTH),
341     .ADDR_SERIAL_BIT_WIDTH(ADDR_SERIAL_BIT_WIDTH),
342     .DATA_SERIAL_BIT_WIDTH(DATA_SERIAL_BIT_WIDTH)
343 )
344 masterInterface_1(
345     .clk(clk),
346     .rst_reset(rst_reset),
347     .reset(reset),
348     .bossRequestIn(boss1RequestIn),
349     .bossAddressIn(boss1AddressIn),
350     .bossAddressValidIn(boss1AddressValidIn),
351     .bossReadIn(boss1ReadIn),
352     .bossWriteIn(boss1WriteIn),
353     .bossDataIn(boss1DataIn),
354     .bossDataValidIn(boss1DataValidIn),
355     .splitIn(master1_split),
356     .grantAccessIn(master1_grant_access),
357     .preSwapIn(preswap_wire),
358     .releasedIn(boss1_released),
359     .dataIn(master1_data_in),
360     .addrReadyIn(addrdecode_ready_wire),
361     .noAddressIn(master1_no_address),
362     .slaveSplitIn(master1_arbitor_slave_split),
363     .noAddressOut(boss1NoAddressOut),
364     .dataAddrReadyOut(boss1DataAddrReadyOut),
365     .masterSplitOut(master1_split_state),
366     .bossDataOut(boss1DataOut),
367     .dataOut(master1_data_out),
368     .requestOut(master1_request),
369     .addrOut(master1_addr_out),
370     .endOut(boss1DataValidOut),
371     .transIncompleteOut(boss1TransIncompleteOut)
372 );
373
374 masterInterface#(
375     .STATE_REG_WIDTH(MASTER_INT_STATE_REG_WIDTH),
376     .DATA_BUS_WIDTH(DATA_BUS_WIDTH),
377     .ADDR_BUS_WIDTH(ADDR_BUS_WIDTH),
378     .ADDR_SERIAL_BIT_WIDTH(ADDR_SERIAL_BIT_WIDTH),
379     .DATA_SERIAL_BIT_WIDTH(DATA_SERIAL_BIT_WIDTH)
380 )
381 masterInterface_2(
382     .clk(clk),
383     .rst_reset(rst_reset),
384     .reset(reset),
385     .bossRequestIn(boss2RequestIn),
386     .bossAddressIn(boss2AddressIn),
387     .bossAddressValidIn(boss2AddressValidIn),
388     .bossReadIn(boss2ReadIn),
389     .bossWriteIn(boss2WriteIn),
390     .bossDataIn(boss2DataIn),
391     .bossDataValidIn(boss2DataValidIn),
392     .splitIn(master2_split),
393     .grantAccessIn(master2_grant_access),
394     .preSwapIn(preswap_wire),
395     .releasedIn(boss2_released),
396     .dataIn(master2_data_in),
397     .addrReadyIn(addrdecode_ready_wire),

```

```

398     .noAddressIn(master2_no_address),
399     .slaveSplitIn(master2_arbitor_slave_split),
400     .noAddressOut(boss2NoAddressOut),
401     .dataAddrReadyOut(boss2DataAddrReadyOut),
402     .masterSplitOut(master2_split_state),
403     .bossDataOut(boss2DataOut),
404     .dataOut(master2_data_out),
405     .requestOut(master2_request),
406     .addrOut(master2_addr_out),
407     .endOut(boss2DataValidOut),
408     .transIncompleteOut(boss2TransIncompleteOut)
409 );
410
411 addrDecode#(
412     .STATE_REG_WIDTH(ADDR_DECODE_STATE_REG_WIDTH),
413     .ADDR_SERIAL_BIT_WIDTH(ADDR_SERIAL_BIT_WIDTH),
414     .ADDR_RANGE_BEGIN(SLAVE1_ADDR_RANGE_BEGIN),
415     .ADDR_RANGE_END(SLAVE1_ADDR_RANGE_END)
416 )
417 addrDecode_1(
418     .clk(clk),
419     .rst_reset(rst_reset),
420     .reset(reset),
421     .addrIn(master2slave_addr),
422     .splitIn(slave1_split),
423     .doneIn(slave1_done),
424     .addrReadyIn(slave1_addr_ready),
425     .addrOut(slave1_addr),
426     .addrValidOut(slave1_addr_valid),
427     .stateRegOut(),
428     .ackSignalOut(),
429     .readyOut(addrdecode1_ready),
430     .slaveHitOut(slave1_hit),
431     .slaveNotHitOut(slave1_not_hit)
432 );
433
434 slaveInterface#(
435     .DATA_BUS_WIDTH(DATA_BUS_WIDTH),
436     .ADDR_BUS_WIDTH(ADDR_BUS_WIDTH),
437     .ADDR_SERIAL_BIT_WIDTH(ADDR_SERIAL_BIT_WIDTH),
438     .DATA_SERIAL_BIT_WIDTH(DATA_SERIAL_BIT_WIDTH)
439 )
440 slaveInterface_1(
441     .clk(clk),
442     .rst_reset(rst_reset),
443     .reset(reset),
444     .dataIn(master2slave_data),
445     .slaveDataIn(slave1DataIn),
446     .slaveDataValidIn(slave1DataValidIn),
447     .addrValidIn(slave1_addr_valid),
448     .addrIn(slave1_addr),
449     .busyIn(slave1BusyIn),
450     .releasedIn(slave1_released),
451     .slaveWriteResponseIn(slave1WriteResponseIn),
452     .dataOut(slave1_data_out),
453     .slaveDataValidOut(slave1DataValidOut),
454     .slaveDataOut(slave1DataOut),
455     .slaveAddrValidOut(slave1AddrValidOut),
456     .slaveAddrOut(slave1AddrOut),
457     .splitOut(slave1_split),
458     .unsplitOut(slave1_unsplit),
459     .slaveReadEnOut(slave1ReadEnOut),
460     .slaveWriteEnOut(slave1WriteEnOut),
461     .slaveDataReadyOut(slave1DataReadyOut),
462     .addrReadyOut(slave1_addr_ready),
463     .addrDoneOut(slave1_done),
464     .preSwapOut(slave1_preswap),
465     .swapGrantOut(slave1_swap_grant)
466 );

```

```

467
468 addrDecode#(
469     .STATE_REG_WIDTH(ADDR_DECODE_STATE_REG_WIDTH),
470     .ADDR_SERIAL_BIT_WIDTH(ADDR_SERIAL_BIT_WIDTH),
471     .ADDR_RANGE_BEGIN(SLAVE2_ADDR_RANGE_BEGIN),
472     .ADDR_RANGE_END(SLAVE2_ADDR_RANGE_END)
473 )
474 addrDecode_2(
475     .clk(clk),
476     .rst_reset(rst_reset),
477     .reset(reset),
478     .addrIn(master2slave_addr),
479     .splitIn(slave2_split),
480     .doneIn(slave2_done),
481     .addrReadyIn(slave2_addr_ready),
482     .addrOut(slave2_addr),
483     .addrValidOut(slave2_addr_valid),
484     .stateRegOut(),
485     .ackSignalOut(),
486     .readyOut(addrdecode2_ready),
487     .slaveHitOut(slave2_hit),
488     .slaveNotHitOut(slave2_not_hit)
489 );
490
491 slaveInterface#(
492     .DATA_BUS_WIDTH(DATA_BUS_WIDTH),
493     .ADDR_BUS_WIDTH(ADDR_BUS_WIDTH),
494     .ADDR_SERIAL_BIT_WIDTH(ADDR_SERIAL_BIT_WIDTH),
495     .DATA_SERIAL_BIT_WIDTH(DATA_SERIAL_BIT_WIDTH)
496 )
497 slaveInterface_2(
498     .clk(clk),
499     .rst_reset(rst_reset),
500     .reset(reset),
501     .dataIn(master2slave_data),
502     .slaveDataIn(slave2DataIn),
503     .slaveDataValidIn(slave2DataValidIn),
504     .addrValidIn(slave2_addr_valid),
505     .addrIn(slave2_addr),
506     .busyIn(slave2BusyIn),
507     .releasedIn(slave2_released),
508     .slaveWriteResponseIn(slave2WriteResponseIn),
509     .dataOut(slave2_data_out),
510     .slaveDataValidOut(slave2DataValidOut),
511     .slaveDataOut(slave2DataOut),
512     .slaveAddrValidOut(slave2AddrValidOut),
513     .slaveAddrOut(slave2AddrOut),
514     .splitOut(slave2_split),
515     .unsplitOut(slave2_unsplit),
516     .slaveReadEnOut(slave2ReadEnOut),
517     .slaveWriteEnOut(slave2WriteEnOut),
518     .slaveDataReadyOut(slave2DataReadyOut),
519     .addrReadyOut(slave2_addr_ready),
520     .addrDoneOut(slave2_done),
521     .preSwapOut(slave2_preswap),
522     .swapGrantOut(slave2_swap_grant)
523 );
524
525 addrDecode#(
526     .STATE_REG_WIDTH(ADDR_DECODE_STATE_REG_WIDTH),
527     .ADDR_SERIAL_BIT_WIDTH(ADDR_SERIAL_BIT_WIDTH),
528     .ADDR_RANGE_BEGIN(SLAVE3_ADDR_RANGE_BEGIN),
529     .ADDR_RANGE_END(SLAVE3_ADDR_RANGE_END)
530 )
531 addrDecode_3(
532     .clk(clk),
533     .rst_reset(rst_reset),
534     .reset(reset),
535     .addrIn(master2slave_addr),

```

```

536     .splitIn(slave3_split),
537     .doneIn(slave3_done),
538     .addrReadyIn(slave3_addr_ready),
539     .addrOut(slave3_addr),
540     .addrValidOut(slave3_addr_valid),
541     .stateRegOut(),
542     .ackSignalOut(),
543     .readyOut(addrdecode3_ready),
544     .slaveHitOut(slave3_hit),
545     .slaveNotHitOut(slave3_not_hit)
546 );
547
548 slaveInterface#(
549     .DATA_BUS_WIDTH(DATA_BUS_WIDTH),
550     .ADDR_BUS_WIDTH(ADDR_BUS_WIDTH),
551     .ADDR_SERIAL_BIT_WIDTH(ADDR_SERIAL_BIT_WIDTH),
552     .DATA_SERIAL_BIT_WIDTH(DATA_SERIAL_BIT_WIDTH)
553 )
554 slaveInterface_3(
555     .clk(clk),
556     .rst_reset(rst_reset),
557     .reset(reset),
558     .dataIn(master2slave_data),
559     .slaveDataIn(slave3DataIn),
560     .slaveDataValidIn(slave3DataValidIn),
561     .addrValidIn(slave3_addr_valid),
562     .addrIn(slave3_addr),
563     .busyIn(slave3BusyIn),
564     .releasedIn(slave3_released),
565     .slaveWriteResponseIn(slave3WriteResponseIn),
566     .dataOut(slave3_data_out),
567     .slaveDataValidOut(slave3DataValidOut),
568     .slaveDataOut(slave3DataOut),
569     .slaveAddrValidOut(slave3AddrValidOut),
570     .slaveAddrOut(slave3AddrOut),
571     .splitOut(slave3_split),
572     .unsplitOut(slave3_unsplit),
573     .slaveReadEnOut(slave3ReadEnOut),
574     .slaveWriteEnOut(slave3WriteEnOut),
575     .slaveDataReadyOut(slave3DataReadyOut),
576     .addrReadyOut(slave3_addr_ready),
577     .addrDoneOut(slave3_done),
578     .preSwapOut(slave3_preswap),
579     .swapGrantOut(slave3_swap_grant)
580 );
581
582 endmodule

```

```

1  `timescale 1ns / 1ps
2  //////////////////////////////////////////////////
3  // Company: ENTC, University of Moratuwa
4  // Author: Miyuru Thathsara | Nadarasar Bahavan
5  //
6  // Create Date: 12/17/2020 02:15:54 AM
7  // Design Name: Bus Arbitor
8  // Module Name: busArbitor
9  // Project Name: System Bus
10 // Additional Comments:
11 //
12 //////////////////////////////////////////////////
13 module busArbitor(
14     clk,
15     rst_reset,
16     reset,
17     requestIn,
18     splitIn,
19     unsplitIn,
20     dismissedIn,
21     masterReleasedOut,
22     slaveReleasedOut,
23     masterSplitStateIn,
24     slaveHitIn,
25     slaveNotHitIn,
26     swapGrantIn,
27     grantAccessOut,
28     mSplitOut,
29     noAddressOut,
30     slaveSplitOut
31 );
32     `include "log.v"
33 //////////////////////////////////////////////////
34 // Parameters
35 //////////////////////////////////////////////////
36 parameter NUM_OF_MASTERS = 2;
37 parameter NUM_OF_SLAVES = 3;
38 //////////////////////////////////////////////////
39 // Local Parameters
40 //////////////////////////////////////////////////
41 localparam GRANT_ACCESS_BIT_WIDTH = 2;
42 localparam CASE_REG_BIT_WIDTH = 3;
43 localparam NUM_OF_STATUS_REGS = NUM_OF_MASTERS;
44 localparam STATUS_REG_WIDTH = NUM_OF_MASTERS
45 + NUM_OF_SLAVES;
46 localparam IDLE_STATE = 3'b000;
47 localparam M1_GRANT_STATE = 3'b001;
48 localparam M2_GRANT_STATE = 3'b011; //There
49 are two grant states.
50 localparam RELEASE_STATE = 3'b010;
51 localparam S1_ADDRESSED = 3'b100;
52 localparam S2_ADDRESSED = 3'b101;
53 localparam S3_ADDRESSED = 3'b110;
54 //////////////////////////////////////////////////
55 // I/O Configuration
56 //////////////////////////////////////////////////
57 input clk;
58 input rst_reset;
59 input reset;
60 input [ NUM_OF_SLAVES - 1 : 0 ] splitIn;
61 input [ NUM_OF_SLAVES - 1 : 0 ] unsplitIn;
62 input [ NUM_OF_SLAVES - 1 : 0 ] dismissedIn;
63 input [ NUM_OF_MASTERS - 1 : 0 ] requestIn;
64 input [ NUM_OF_MASTERS - 1 : 0 ] masterSplitStateIn;
65 input [ NUM_OF_SLAVES - 1 : 0 ] slaveHitIn;
66 input [ NUM_OF_SLAVES - 1 : 0 ] slaveNotHitIn;
67 input [ NUM_OF_SLAVES - 1 : 0 ] swapGrantIn;
68 output reg [ NUM_OF_MASTERS - 1 : 0 ] masterReleasedOut;
69 output reg [ NUM_OF_SLAVES - 1 : 0 ] slaveReleasedOut;

```

```

68 output reg      [ NUM_OF_MASTERS - 1 : 0 ]      grantAccessOut;
69 output reg      [ NUM_OF_MASTERS - 1 : 0 ]      mSplitOut;
70 output reg      [ NUM_OF_MASTERS - 1 : 0 ]      noAddressOut;
71 output reg      [ NUM_OF_MASTERS - 1 : 0 ]      slaveSplitOut;
72 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
73 // Wires and Registers
74 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
75 reg              [ CASE_REG_BIT_WIDTH - 1 : 0 ]      case_reg;
76 reg              reset_a;
77 reg              reset_b;
78 reg              [ STATUS_REG_WIDTH - 1 : 0 ]      status_reg1;
79 reg              [ STATUS_REG_WIDTH - 1 : 0 ]      status_reg2;
80 reg              [ NUM_OF_STATUS_REGS - 1 : 0 ]      occupancy_status_regs;
81 reg              [ NUM_OF_MASTERS - 1 : 0 ]      master_grant_status;
82 reg              swap_given_flag;
83 reg              released_flag;
84 reg              [ NUM_OF_SLAVES - 1 : 0 ]      unsplit_state;
85 wire            reset_wire;
86 //////////////////////////////////////////////////////////////////Priorities
87 // M1 has higher priority than M2
88 // M1 corresponds to [1,0]
89 // M2 corresponds to [0,1]
90 //This is important for the signals: RequestIn, masterSplitStateIn,grantAccessOut
91 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
92 // Implementation
93 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
94 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
95
96 assign reset_wire                                = reset |
reset_b;
97
98 always@(posedge clk)begin
99     if(~rst_reset)begin
100         reset_a                                <= 1'b0;
101         reset_b                                <= 1'b0;
102     end
103     else begin
104         reset_a                                <= reset;
105         reset_b                                <= reset_a;
106     end
107 end
108
109 always@(posedge clk or negedge reset_wire)begin
110     if(~reset_wire)begin
111         case_reg                                <= IDLE_STATE;
112         status_reg1                             <= {
113             STATUS_REG_WIDTH {1'b0} };
114         status_reg2                             <= {
115             STATUS_REG_WIDTH {1'b0} };
116         occupancy_status_regs                   <= {
117             NUM_OF_STATUS_REGS {1'b0} };
118         master_grant_status                     <= {
119             NUM_OF_MASTERS {1'b0} };
120         swap_given_flag                         <= 1'b0;
121         released_flag                           <= 1'b0;
122         mSplitOut                              <= {
123             NUM_OF_MASTERS {1'b0} };
124         unsplit_state                           <= {
125             NUM_OF_SLAVES {1'b0} };
126         masterReleasedOut                       <= {
127             NUM_OF_MASTERS {1'b0} };
128         slaveReleasedOut                       <= {
129             NUM_OF_SLAVES {1'b0} };
130         noAddressOut                           <= {
131             NUM_OF_MASTERS {1'b0} };
132         slaveSplitOut                          <= {
133             NUM_OF_MASTERS {1'b0} };

```

```

124     end
125     else begin
126         case(case_reg)
127             IDLE_STATE:
128                 begin
129                     slaveSplitOut                                     <= {
130                     NUM_OF_MASTERS {1'b0} };
131                     if(!unsplitIn) begin
132                         grantAccessOut                               <= 2'b00;
133                         unsplit_state                               <= unsplitIn;
134                         case_reg                                     <= RELEASE_STATE;
135                         masterReleasedOut                           <= {
136                         NUM_OF_MASTERS {1'b0} };
137                         slaveReleasedOut                           <= {
138                         NUM_OF_SLAVES {1'b0} };
139                     end
140                     else begin
141                         noAddressOut                                 <= {
142                         NUM_OF_MASTERS {1'b0} };
143                         if (requestIn[1]) begin
144                             grantAccessOut                         <= 2'b10;
145                             master_grant_status                   <= 2'b10;
146                             case_reg                             <=
147                             M1_GRANT_STATE;
148                         end
149                         else begin
150                             if (requestIn[0]) begin
151                                 grantAccessOut                     <= 2'b01;
152                                 master_grant_status                 <= 2'b01;
153                                 case_reg                           <=
154                                 M2_GRANT_STATE;
155                             end
156                             else begin
157                                 grantAccessOut                     <= 2'b00;
158                                 master_grant_status                 <= 2'b00;
159                                 case_reg                           <= IDLE_STATE;
160                             end
161                         end
162                     end
163                 end
164             M1_GRANT_STATE:
165                 begin
166                     masterReleasedOut                               <= {
167                     NUM_OF_MASTERS {1'b0} };
168                     slaveReleasedOut                               <= {
169                     NUM_OF_SLAVES {1'b0} };
170                     if(swap_given_flag)begin
171                         if(dismissedIn)begin
172                             status_reg1                           <= {
173                             STATUS_REG_WIDTH {1'b0} };
174                             occupancy_status_regs[1]               <= 1'b0;
175                             swap_given_flag                       <= 1'b0;
176                             master_grant_status                   <= {
177                             NUM_OF_MASTERS {1'b0} };
178                             case_reg                               <= IDLE_STATE;
179                         end
180                         else begin
181                             swap_given_flag                       <= 1'b1;
182                             case_reg                               <=
183                             M1_GRANT_STATE;
184                         end
185                     end
186                     else begin
187                         if(released_flag)begin
188                             if(dismissedIn)begin
189                                 status_reg1                           <= {
190                                 STATUS_REG_WIDTH {1'b0} };
191                                 occupancy_status_regs[1]               <= 1'b0;
192                                 released_flag                       <= 1'b0;

```

```

181         master_grant_status                <= {
182         NUM_OF_MASTERS {1'b0} };
183         case_reg                            <= IDLE_STATE;
184     end
185     else begin
186         released_flag                        <= 1'b1;
187         case_reg                            <=
188         M1_GRANT_STATE;
189     end
190     end
191     else begin
192         if(!slaveHitIn)begin
193             case(slaveHitIn)
194                 3'b100:
195                 begin
196                     if(occupancy_status_regs[0])begin
197                         if(status_reg2[ NUM_OF_SLAVES - 1 : 0 ] ==
198                         3'b100)begin
199                             case_reg                <= IDLE_STATE;
200                             master_grant_status      <= {
201                             NUM_OF_MASTERS {1'b0} };
202                             grantAccessOut           <= {
203                             NUM_OF_MASTERS {1'b0} };
204                             slaveSplitOut           <= 2'b10;
205                         end
206                     else begin
207                         case_reg                <= S1_ADDRESSED;
208                         status_reg1             <= {
209                         master_grant_status, 3'b100 };
210                         occupancy_status_regs[1]   <= 1'b1;
211                     end
212                 end
213             else begin
214                 case_reg                <= S1_ADDRESSED;
215                 status_reg1             <= {
216                 master_grant_status, 3'b100 };
217                 occupancy_status_regs[1]   <= 1'b1;
218             end
219         end
220         3'b010:
221         begin
222             if(occupancy_status_regs[0])begin
223                 if(status_reg2[ NUM_OF_SLAVES - 1 : 0 ] ==
224                 3'b010)begin
225                     case_reg                <= IDLE_STATE;
226                     master_grant_status      <= {
227                     NUM_OF_MASTERS {1'b0} };
228                     grantAccessOut           <= {
229                     NUM_OF_MASTERS {1'b0} };
230                     slaveSplitOut           <= 2'b10;
231                 end
232             else begin
233                 case_reg                <= S2_ADDRESSED;
234                 status_reg1             <= {
235                 master_grant_status, 3'b010 };
236                 occupancy_status_regs[1]   <= 1'b1;
237             end
238         end
239     else begin
240         case_reg                <= S2_ADDRESSED;
241         status_reg1             <= {
242         master_grant_status, 3'b010 };
243         occupancy_status_regs[1] <= 1'b1;
244     end
245 end
246 3'b001:
247 begin
248     if(occupancy_status_regs[0])begin
249         if(status_reg2[ NUM_OF_SLAVES - 1 : 0 ] ==

```

```

238         3'b001)begin
239             case_reg                                     <= IDLE_STATE;
240             master_grant_status                         <= {
241                 NUM_OF_MASTERS {1'b0} };
242             grantAccessOut                             <= {
243                 NUM_OF_MASTERS {1'b0} };
244             slaveSplitOut                             <= 2'b10;
245         end
246     else begin
247         case_reg                                     <= S3_ADDRESSED;
248         status_reg1                                  <= {
249             master_grant_status, 3'b001 };
250         occupancy_status_regs[1]                    <= 1'b1;
251     end
252 end
253 else begin
254     case_reg                                     <= S3_ADDRESSED;
255     status_reg1                                  <= {
256         master_grant_status, 3'b001 };
257     occupancy_status_regs[1]                    <= 1'b1;
258 end
259 end
260 default begin
261     case_reg                                     <=
262     M1_GRANT_STATE;
263 end
264 endcase
265 end
266 else begin
267     if(&slaveNotHitIn)begin
268         noAddressOut                             <= 2'b10;
269         master_grant_status                     <= {
270             NUM_OF_MASTERS {1'b0} };
271         case_reg                             <= IDLE_STATE;
272     end
273     else begin
274         case_reg                             <=
275         M1_GRANT_STATE;
276     end
277 end
278 end
279 end
280 M2_GRANT_STATE:
281 begin
282     masterReleasedOut                             <= {
283         NUM_OF_MASTERS {1'b0} };
284     slaveReleasedOut                             <= {
285         NUM_OF_SLAVES {1'b0} };
286     if(swap_given_flag)begin
287         if(dismissedIn)begin
288             status_reg2                         <= {
289                 STATUS_REG_WIDTH {1'b0} };
290             occupancy_status_regs[0]           <= 1'b0;
291             swap_given_flag                     <= 1'b0;
292             master_grant_status                 <= {
293                 NUM_OF_MASTERS {1'b0} };
294             case_reg                             <= IDLE_STATE;
295         end
296         else begin
297             swap_given_flag                     <= 1'b1;
298             case_reg                             <=
299             M2_GRANT_STATE;
300         end
301     end
302 end
303 else begin
304     if(released_flag)begin
305         if(dismissedIn)begin
306             status_reg2                         <= {

```

```

STATUS_REG_WIDTH {1'b0} };
294 occupancy_status_regs[0] <= 1'b0;
295 released_flag <= 1'b0;
296 master_grant_status <= {
NUM_OF_MASTERS {1'b0} };
case_reg <= IDLE_STATE;
297
298 end
299 else begin
300     released_flag <= 1'b1;
301     case_reg <=
M2_GRANT_STATE;
end
302
303 end
304 else begin
305     if(!slaveHitIn)begin
306         case(slaveHitIn)
307             3'b100:
308                 begin
309                     if(occupancy_status_regs[1])begin
310                         if(status_reg1[ NUM_OF_SLAVES - 1 : 0 ] ==
311                             3'b100)begin
312                             case_reg <= IDLE_STATE;
313                             master_grant_status <= {
NUM_OF_MASTERS {1'b0} };
grantAccessOut <= {
NUM_OF_MASTERS {1'b0} };
slaveSplitOut <= 2'b01;
end
314
315                         else begin
316                             case_reg <= S1_ADDRESSED;
317                             status_reg2 <= {
master_grant_status, 3'b100 };
occupancy_status_regs[0] <= 1'b1;
end
318
319                     end
320                 end
321             end
322         else begin
323             case_reg <= S1_ADDRESSED;
324             status_reg2 <= {
master_grant_status, 3'b100 };
occupancy_status_regs[0] <= 1'b1;
end
325
326         end
327     end
328     3'b010:
329     begin
330         if(occupancy_status_regs[1])begin
331             if(status_reg1[ NUM_OF_SLAVES - 1 : 0 ] ==
332                 3'b010)begin
333                 case_reg <= IDLE_STATE;
334                 master_grant_status <= {
NUM_OF_MASTERS {1'b0} };
grantAccessOut <= {
NUM_OF_MASTERS {1'b0} };
slaveSplitOut <= 2'b01;
end
335
336             else begin
337                 case_reg <= S2_ADDRESSED;
338                 status_reg2 <= {
master_grant_status, 3'b010 };
occupancy_status_regs[0] <= 1'b1;
end
339
340             end
341         else begin
342             case_reg <= S2_ADDRESSED;
343             status_reg2 <= {
master_grant_status, 3'b010 };
occupancy_status_regs[0] <= 1'b1;
end
344
345         end
346     end
347     3'b001:

```

```

350         begin
351             if(occupancy_status_regs[1])begin
352                 if(status_reg1[ NUM_OF_SLAVES - 1 : 0 ] ==
353                     3'b001)begin
354                     case_reg                <= IDLE_STATE;
355                     master_grant_status      <= {
356                         NUM_OF_MASTERS {1'b0} };
357                     grantAccessOut          <= {
358                         NUM_OF_MASTERS {1'b0} };
359                     slaveSplitOut           <= 2'b01;
360                 end
361             else begin
362                 case_reg                <= S3_ADDRESSED;
363                 status_reg2            <= {
364                     master_grant_status, 3'b001 };
365                 occupancy_status_regs[0] <= 1'b1;
366             end
367         end
368     else begin
369         case_reg                <= S3_ADDRESSED;
370         status_reg2            <= {
371             master_grant_status, 3'b001 };
372         occupancy_status_regs[0] <= 1'b1;
373     end
374 end
375 default begin
376     case_reg                <=
377     M2_GRANT_STATE;
378 end
379 endcase
380 end
381 else begin
382     if(&slaveNotHitIn)begin
383         noAddressOut          <= 2'b01;
384         master_grant_status    <= {
385             NUM_OF_MASTERS {1'b0} };
386         case_reg              <= IDLE_STATE;
387     end
388     else begin
389         case_reg              <=
390         M2_GRANT_STATE;
391     end
392 end
393 end
394 end
395 end
396 end
397 end
398 end
399 end
400 end
401 end
402 end
403 end
404 end
405 end
406 end
407 end
408 end
409 end
410 end
411 end
412 end
413 end
414 end
415 end
416 end
417 end
418 end
419 end
420 end
421 end
422 end
423 end
424 end
425 end
426 end
427 end
428 end
429 end
430 end
431 end
432 end
433 end
434 end
435 end
436 end
437 end
438 end
439 end
440 end
441 end
442 end
443 end
444 end
445 end
446 end
447 end
448 end
449 end
450 end
451 end
452 end
453 end
454 end
455 end
456 end
457 end
458 end
459 end
460 end
461 end
462 end
463 end
464 end
465 end
466 end
467 end
468 end
469 end
470 end
471 end
472 end
473 end
474 end
475 end
476 end
477 end
478 end
479 end
480 end
481 end
482 end
483 end
484 end
485 end
486 end
487 end
488 end
489 end
490 end
491 end
492 end
493 end
494 end
495 end
496 end
497 end
498 end
499 end
500 end
501 end
502 end
503 end
504 end
505 end
506 end
507 end
508 end
509 end
510 end
511 end
512 end
513 end
514 end
515 end
516 end
517 end
518 end
519 end
520 end
521 end
522 end
523 end
524 end
525 end
526 end
527 end
528 end
529 end
530 end
531 end
532 end
533 end
534 end
535 end
536 end
537 end
538 end
539 end
540 end
541 end
542 end
543 end
544 end
545 end
546 end
547 end
548 end
549 end
550 end
551 end
552 end
553 end
554 end
555 end
556 end
557 end
558 end
559 end
560 end
561 end
562 end
563 end
564 end
565 end
566 end
567 end
568 end
569 end
570 end
571 end
572 end
573 end
574 end
575 end
576 end
577 end
578 end
579 end
580 end
581 end
582 end
583 end
584 end
585 end
586 end
587 end
588 end
589 end
590 end
591 end
592 end
593 end
594 end
595 end
596 end
597 end
598 end
599 end
600 end
601 end
602 end
603 end
604 end
605 end
606 end
607 end
608 end
609 end
610 end
611 end
612 end
613 end
614 end
615 end
616 end
617 end
618 end
619 end
620 end
621 end
622 end
623 end
624 end
625 end
626 end
627 end
628 end
629 end
630 end
631 end
632 end
633 end
634 end
635 end
636 end
637 end
638 end
639 end
640 end
641 end
642 end
643 end
644 end
645 end
646 end
647 end
648 end
649 end
650 end
651 end
652 end
653 end
654 end
655 end
656 end
657 end
658 end
659 end
660 end
661 end
662 end
663 end
664 end
665 end
666 end
667 end
668 end
669 end
670 end
671 end
672 end
673 end
674 end
675 end
676 end
677 end
678 end
679 end
680 end
681 end
682 end
683 end
684 end
685 end
686 end
687 end
688 end
689 end
690 end
691 end
692 end
693 end
694 end
695 end
696 end
697 end
698 end
699 end
700 end
701 end
702 end
703 end
704 end
705 end
706 end
707 end
708 end
709 end
710 end
711 end
712 end
713 end
714 end
715 end
716 end
717 end
718 end
719 end
720 end
721 end
722 end
723 end
724 end
725 end
726 end
727 end
728 end
729 end
730 end
731 end
732 end
733 end
734 end
735 end
736 end
737 end
738 end
739 end
740 end
741 end
742 end
743 end
744 end
745 end
746 end
747 end
748 end
749 end
750 end
751 end
752 end
753 end
754 end
755 end
756 end
757 end
758 end
759 end
760 end
761 end
762 end
763 end
764 end
765 end
766 end
767 end
768 end
769 end
770 end
771 end
772 end
773 end
774 end
775 end
776 end
777 end
778 end
779 end
780 end
781 end
782 end
783 end
784 end
785 end
786 end
787 end
788 end
789 end
790 end
791 end
792 end
793 end
794 end
795 end
796 end
797 end
798 end
799 end
800 end
801 end
802 end
803 end
804 end
805 end
806 end
807 end
808 end
809 end
810 end
811 end
812 end
813 end
814 end
815 end
816 end
817 end
818 end
819 end
820 end
821 end
822 end
823 end
824 end
825 end
826 end
827 end
828 end
829 end
830 end
831 end
832 end
833 end
834 end
835 end
836 end
837 end
838 end
839 end
840 end
841 end
842 end
843 end
844 end
845 end
846 end
847 end
848 end
849 end
850 end
851 end
852 end
853 end
854 end
855 end
856 end
857 end
858 end
859 end
860 end
861 end
862 end
863 end
864 end
865 end
866 end
867 end
868 end
869 end
870 end
871 end
872 end
873 end
874 end
875 end
876 end
877 end
878 end
879 end
880 end
881 end
882 end
883 end
884 end
885 end
886 end
887 end
888 end
889 end
890 end
891 end
892 end
893 end
894 end
895 end
896 end
897 end
898 end
899 end
900 end
901 end
902 end
903 end
904 end
905 end
906 end
907 end
908 end
909 end
910 end
911 end
912 end
913 end
914 end
915 end
916 end
917 end
918 end
919 end
920 end
921 end
922 end
923 end
924 end
925 end
926 end
927 end
928 end
929 end
930 end
931 end
932 end
933 end
934 end
935 end
936 end
937 end
938 end
939 end
940 end
941 end
942 end
943 end
944 end
945 end
946 end
947 end
948 end
949 end
950 end
951 end
952 end
953 end
954 end
955 end
956 end
957 end
958 end
959 end
960 end
961 end
962 end
963 end
964 end
965 end
966 end
967 end
968 end
969 end
970 end
971 end
972 end
973 end
974 end
975 end
976 end
977 end
978 end
979 end
980 end
981 end
982 end
983 end
984 end
985 end
986 end
987 end
988 end
989 end
990 end
991 end
992 end
993 end
994 end
995 end
996 end
997 end
998 end
999 end
1000 end

```

```

410         slaveReleasedOut                                     <= 3'b010;
411         mSplitOut[1]                                         <= 1'b0;
412         grantAccessOut                                       <= 2'b10;
413         released_flag                                         <= 1'b1;
414         case_reg                                              <=
415                                     M1_GRANT_STATE;
416     end
417     else begin
418         case_reg                                              <= RELEASE_STATE;
419     end
420 end
421 3'b001:
422 begin
423     if(unsplit_state[0] == 1)begin
424         masterReleasedOut                                     <= 2'b10;
425         slaveReleasedOut                                     <= 3'b001;
426         mSplitOut[1]                                         <= 1'b0;
427         grantAccessOut                                       <= 2'b10;
428         released_flag                                         <= 1'b1;
429         case_reg                                              <=
430                                     M1_GRANT_STATE;
431     end
432     else begin
433         case_reg                                              <= RELEASE_STATE;
434     end
435 endcase
436 end
437 else begin
438     if(occupancy_status_regs[0])begin
439         case(status_reg2[ NUM_OF_SLAVES - 1 : 0 ])
440             3'b100:
441             begin
442                 if(unsplit_state[2] == 1)begin
443                     masterReleasedOut                         <= 2'b01;
444                     slaveReleasedOut                         <= 3'b100;
445                     mSplitOut[0]                             <= 1'b0;
446                     grantAccessOut                           <= 2'b01;
447                     released_flag                             <= 1'b1;
448                     case_reg                                  <=
449                                     M2_GRANT_STATE;
450                 end
451                 else begin
452                     case_reg                                  <= RELEASE_STATE;
453                 end
454             end
455             3'b010:
456             begin
457                 if(unsplit_state[1] == 1)begin
458                     masterReleasedOut                         <= 2'b01;
459                     slaveReleasedOut                         <= 3'b010;
460                     mSplitOut[0]                             <= 1'b0;
461                     grantAccessOut                           <= 2'b01;
462                     released_flag                             <= 1'b1;
463                     case_reg                                  <=
464                                     M2_GRANT_STATE;
465                 end
466                 else begin
467                     case_reg                                  <= RELEASE_STATE;
468                 end
469             end
470             3'b001:
471             begin
472                 if(unsplit_state[0] == 1)begin
473                     masterReleasedOut                         <= 2'b01;
474                     slaveReleasedOut                         <= 3'b001;
475                     mSplitOut[0]                             <= 1'b0;
476                     grantAccessOut                           <= 2'b01;
477                     released_flag                             <= 1'b1;

```

```

475             case_reg                                     <=
476             M2_GRANT_STATE;
477         end
478         else begin
479             case_reg                                     <= RELEASE_STATE;
480         end
481     endcase
482 end
483 end
484 end
485 S1_ADDRESSED:
486 begin
487     if(swapGrantIn[2])begin
488         swap_given_flag                                     <= 1'b1;
489         if(swapGrantIn == status_reg1[ NUM_OF_SLAVES - 1 : 0 ])begin
490             status_reg1                                     <= {
491                 STATUS_REG_WIDTH {1'b0} };
492             occupancy_status_regs[1]                       <= 1'b0;
493             case_reg                                       <=
494                 M1_GRANT_STATE;
495         end
496         else begin
497             if(swapGrantIn == status_reg2[ NUM_OF_SLAVES - 1 : 0 ])begin
498                 status_reg2                                 <= {
499                     STATUS_REG_WIDTH {1'b0} };
500                 occupancy_status_regs[0]                   <= 1'b0;
501                 case_reg                                   <=
502                     M2_GRANT_STATE;
503             end
504             else begin
505                 case_reg                                     <= S1_ADDRESSED;
506             end
507         end
508     end
509     else begin
510         if(splitIn[2])begin
511             if(splitIn[2] == status_reg1[2])begin
512                 mSplitOut[1]                               <= 1'b1;
513                 master_grant_status                         <= {
514                     NUM_OF_MASTERS {1'b0} };
515                 case_reg                                     <= IDLE_STATE;
516             end
517             else begin
518                 if(splitIn[2] == status_reg2[2])begin
519                     mSplitOut[0]                           <= 1'b1;
520                     master_grant_status                     <= {
521                         NUM_OF_MASTERS {1'b0} };
522                     case_reg                                 <= IDLE_STATE;
523                 end
524                 else begin
525                     case_reg                                 <= S1_ADDRESSED;
526                 end
527             end
528         end
529         else begin
530             case_reg                                       <= S1_ADDRESSED;
531         end
532     end
533 end
534 S2_ADDRESSED:
535 begin
536     if(swapGrantIn[1])begin
537         swap_given_flag                                     <= 1'b1;
538         if(swapGrantIn == status_reg1[ NUM_OF_SLAVES - 1 : 0 ])begin
539             status_reg1                                     <= {
540                 STATUS_REG_WIDTH {1'b0} };
541             occupancy_status_regs[1]                       <= 1'b0;
542             case_reg                                       <=

```

```

536         M1_GRANT_STATE;
537     end
538     else begin
539         if(swapGrantIn == status_reg2[ NUM_OF_SLAVES - 1 : 0 ])begin
540             status_reg2                                     <= {
541                 STATUS_REG_WIDTH {1'b0} };
542             occupancy_status_regs[0]                         <= 1'b0;
543             case_reg                                         <=
544                 M2_GRANT_STATE;
545         end
546     end
547     end
548     else begin
549         if(splitIn[1])begin
550             if(splitIn[1] == status_reg1[1])begin
551                 mSplitOut[1]                                <= 1'b1;
552                 master_grant_status                         <= {
553                     NUM_OF_MASTERS {1'b0} };
554                 case_reg                                     <= IDLE_STATE;
555             end
556             else begin
557                 if(splitIn[1] == status_reg2[1])begin
558                     mSplitOut[0]                            <= 1'b1;
559                     master_grant_status                     <= {
560                         NUM_OF_MASTERS {1'b0} };
561                     case_reg                                <= IDLE_STATE;
562                 end
563                 else begin
564                     case_reg                                 <= S2_ADDRESSED;
565                 end
566             end
567         end
568     end
569     end
570     end
571     S3_ADDRESSED:
572     begin
573         if(swapGrantIn[0])begin
574             swap_given_flag                                  <= 1'b1;
575             if(swapGrantIn == status_reg1[ NUM_OF_SLAVES - 1 : 0 ])begin
576                 status_reg1                                 <= {
577                     STATUS_REG_WIDTH {1'b0} };
578                 occupancy_status_regs[1]                   <= 1'b0;
579                 case_reg                                     <=
580                     M1_GRANT_STATE;
581             end
582             else begin
583                 if(swapGrantIn == status_reg2[ NUM_OF_SLAVES - 1 : 0 ])begin
584                     status_reg2                             <= {
585                         STATUS_REG_WIDTH {1'b0} };
586                     occupancy_status_regs[0]                 <= 1'b0;
587                     case_reg                                 <=
588                         M2_GRANT_STATE;
589                 end
590                 else begin
591                     case_reg                                 <= S3_ADDRESSED;
592                 end
593             end
594         end
595     end
596     else begin
597         if(splitIn[0])begin
598             if(splitIn[0] == status_reg1[0])begin
599                 mSplitOut[1]                                <= 1'b1;
600                 master_grant_status                         <= {

```

```

596         NUM_OF_MASTERS {1'b0} };
597         case_reg
598     end
599     else begin
600         if(splitIn[0] == status_reg2[0])begin
601             mSplitOut[0]
602             master_grant_status
603             NUM_OF_MASTERS {1'b0} };
604             case_reg
605         end
606         else begin
607             case_reg
608         end
609     end
610     else begin
611         case_reg
612     end
613     end
614     default:
615     begin
616         case_reg
617     end
618 endcase
619 end
620 end
621 endmodule
622

```

```

<= IDLE_STATE;
<= 1'b1;
<= {
<= IDLE_STATE;
<= S3_ADDRESSED;
<= S3_ADDRESSED;
<= IDLE_STATE;

```

```
1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company: ENTC, University of Moratuwa
4  // Author: Miyuru Thathsara | Nadarasar Bahavan
5  //
6  // Create Date: 12/17/2020 12:45:05 AM
7  // Design Name: Number of Bits calc
8  // Module Name: log
9  // Project Name: System Bus
10 // Additional Comments:
11 //
12 ///////////////////////////////////////////////////////////////////
13
14 function integer count2width(input integer value);
15     if (value <= 1) begin
16         count2width = 1;
17     end
18     else begin
19         value = value - 1;
20         for (count2width = 0; value > 0; count2width = count2width + 1) begin
21             value = value >> 1;
22         end
23     end
24 endfunction
25
```

```

1  `timescale 1ns / 1ps
2  //////////////////////////////////////////////////
3  // Company: ENTC, University of Moratuwa
4  // Author:  Miyuru Thathsara | Nadarasar Bahavan
5  //
6  // Create Date: 12/16/2020 04:51:49 PM
7  // Design Name: Address Decoder
8  // Module Name: addrDecode
9  // Project Name: System Bus
10 // Additional Comments:
11 //
12 //////////////////////////////////////////////////
13 module masterInterface(
14     clk,
15     rst_reset,
16     reset,
17     bossRequestIn,
18     bossAddressIn,
19     bossAddressValidIn,
20     bossReadIn,
21     bossWriteIn,
22     bossDataIn,
23     bossDataValidIn,
24     splitIn,
25     grantAccessIn,
26     preSwapIn,
27     releasedIn,
28     dataIn,
29     addrReadyIn,
30     noAddressIn,
31     slaveSplitIn,
32     noAddressOut,
33     dataAddrReadyOut,
34     masterSplitOut,
35     bossDataOut,
36     dataOut,
37     requestOut,
38     addrOut,
39     endOut,
40     transIncompleteOut
41 );
42 `include "log.v"
43 //////////////////////////////////////////////////
44 // Parameters
45 //////////////////////////////////////////////////
46 parameter STATE_REG_WIDTH           = 3;
47 parameter DATA_BUS_WIDTH           = 16;
48 parameter ADDR_BUS_WIDTH             = 16;
49 parameter ADDR_SERIAL_BIT_WIDTH      = 19;
50 parameter DATA_SERIAL_BIT_WIDTH     = 17;
51 //////////////////////////////////////////////////
52 // Local Parameters
53 //////////////////////////////////////////////////
54 localparam CASE_REG_BIT_WIDTH        = 3;
55 localparam IDLE_STATE                 = 3'b000;
56 localparam REQUESTING_STATE           = 3'b001;
57 localparam ADDRESSING_STATE           = 3'b010;
58 localparam JUNCTION_STATE             = 3'b011;
59 localparam SPLITROOM_STATE            = 3'b100;
60 localparam SWAP_STATE                  = 3'b110;
61 localparam COMPLETE                   = 3'b111;
62 localparam ADDR_COUNTER_WIDTH         =
count2width(ADDR_SERIAL_BIT_WIDTH);
63 localparam DATA_COUNTER_WIDTH        =
count2width(DATA_SERIAL_BIT_WIDTH);
64 //////////////////////////////////////////////////
65 // I/O Configuration
66 //////////////////////////////////////////////////
67 input                                clk;

```

```

68  input                                rst_reset;
69  input                                reset;
70  // Communication with the Module Side
71  input                                bossRequestIn; // Boss
    Makes a request here using this. It pulls it up.
72  input      [ ADDR_BUS_WIDTH - 1 : 0 ] bossAddressIn; //Boss
    loads the 'Address' of the Slave here. it is of the format (1)(01)(00000) ->
    (Activate) (R) (W) (Data-5bit)
73  input                                bossAddressValidIn;
74  input                                bossReadIn;
75  input                                bossWriteIn;
76  input      [ DATA_BUS_WIDTH - 1 : 0 ] bossDataIn; //If the
    Boss wants a 'wr_wire' operation done, it will send the data in here.
77  input                                bossDataValidIn;
78  input                                grantAccessIn;
79  input                                preSwapIn;
80  input                                releasedIn; //Message
    from Arbiter indicating the master has been released.
81  input                                splitIn;
82  input                                dataIn;
83  input                                addrReadyIn;
84  input                                noAddressIn;
85  input                                slaveSplitIn;
86  output reg                          noAddressOut;
87  output reg                          dataAddrReadyOut;
88  output reg                          requestOut;
89  //Interfaces from the Slave side. (SPLIT OPs)
90  output reg                          masterSplitOut; //(You
    are indicating the bus that you are in a SPLIT state)
91  output                                dataOut;
92  output      [ DATA_BUS_WIDTH - 1 : 0 ] bossDataOut;
93  output                                addrOut;
94  output reg                          endOut;
95  output reg                          transIncompleteOut;
96  //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
97  // Wires & Registers
98  //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
99  //Interfaces to send the data //////////////////////////////////////////////////////////////////Lot's of Chopping
    around here.
100 reg      [ DATA_SERIAL_BIT_WIDTH - 1 : 0 ] data_shift;
101 // Interfaces between Module and Master. Mainly Shift registers
102 reg      [ ADDR_SERIAL_BIT_WIDTH - 1 : 0 ] addr_shift;
103 reg                                              master_busy_state; //
    You give this signal out to the boss to indicate the master is available.
104 reg      [ ADDR_COUNTER_WIDTH - 1 : 0 ] addr_shift_counter;
105 reg      [ DATA_COUNTER_WIDTH - 1 : 0 ] data_counter;
106 reg      [ STATE_REG_WIDTH - 1 : 0 ] case_reg;
107 reg                                              reset_a;
108 reg                                              reset_b;
109 wire                                              reset_wire;
110 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
111 // Implementation
112 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
113
114 assign dataOut                                = data_shift[
    DATA_SERIAL_BIT_WIDTH - 1 ];
115 assign addrOut                                = addr_shift[
    ADDR_SERIAL_BIT_WIDTH - 1 ];
116 assign reset_wire                            = reset | reset_b;
117 assign bossDataOut                          = data_shift[
    DATA_SERIAL_BIT_WIDTH - 2 : 0 ];
118
119 always@(posedge clk)begin
120     if(~rst_reset)begin
121         reset_a                                <= 1'b0;
122         reset_b                                <= 1'b0;
123     end
124     else begin
125         reset_a                                <= reset;

```

```

126         reset_b                                     <= reset_a;
127     end
128 end
129
130 always@(posedge clk or negedge reset_wire)begin
131     if(~reset_wire)begin
132         case_reg                                     <= IDLE_STATE;
133         addr_shift_counter                           <= { ADDR_COUNTER_WIDTH
134             {1'b0} };
135         data_counter                                 <= { DATA_COUNTER_WIDTH
136             {1'b0} };
137         data_shift                                    <= {
138             DATA_SERIAL_BIT_WIDTH {1'b0} };
139         masterSplitOut                               <= 1'b0;
140         dataAddrReadyOut                             <= 1'b0;
141         noAddressOut                                 <= 1'b0;
142         transIncompleteOut                           <= 1'b0;
143     end
144     else begin
145         case(case_reg)
146             IDLE_STATE:
147             begin
148                 transIncompleteOut                   <= 1'b0;
149                 noAddressOut                         <= 1'b0;
150                 if(bossRequestIn)begin //When the Boss Requests something, it will
151                     make the bus go into a requesting state.
152                     case_reg                         <= REQUESTING_STATE;
153                     requestOut                       <= 1'b1; //Using this
154                         signal it will request the arbiter for permission to use the
155                         bus.
156                 end
157             else begin
158                 endOut                               <= 1'b0;
159                 master_busy_state                   <= 1'b0; //Master is
160                     not busy at the moment. Once he moves to any state other than Idle.
161                     He will become busy.
162                 addr_shift                           <= {
163                     ADDR_SERIAL_BIT_WIDTH {1'b0} };
164                 requestOut                           <= 1'b0;
165             end
166             REQUESTING_STATE:
167             begin
168                 //In this state, it will continuously ask the arbiter for permission,
169                 upoun granting, it will load the address into the shift.
170                 if(grantAccessIn)begin
171                     dataAddrReadyOut                 <= 1'b1;
172                     requestOut                       <= 1'b0;
173                     case_reg                         <= ADDRESSING_STATE;
174                     data_shift                       <= { 1'b1, bossDataIn };
175                     addr_shift                       <= { 1'b1, bossReadIn,
176                         bossWriteIn, bossAddressIn }; //Load the address into the Shift
177                         register.
178                     master_busy_state                 <= 1'b1; //Boss request
179                         is now going to be processed, so Master becomes busy.
180                 end
181             else begin
182                 dataAddrReadyOut                     <= 1'b0;
183                 case_reg                             <= REQUESTING_STATE;
184             end
185             ADDRESSING_STATE:
186             begin
187                 dataAddrReadyOut                     <= 1'b0;
188                 if(addrReadyIn)begin
189                     if(addr_shift_counter < ADDR_SERIAL_BIT_WIDTH + 1)begin
190                         addr_shift                   <= addr_shift << 1;
191                         addr_shift_counter            <= addr_shift_counter +
192                             1'b1;

```

```

181         case_reg                                     <= ADDRESSING_STATE;
182     end
183     else begin
184         addr_shift_counter                             <= { ADDR_COUNTER_WIDTH
185             {1'b0} };
186         addr_shift                                     <= {
187             ADDR_SERIAL_BIT_WIDTH {1'b0} };
188         case_reg                                       <= JUNCTION_STATE;
189     end
190 end
191 else begin
192     case_reg                                           <= ADDRESSING_STATE;
193 end
194 end
195 JUNCTION_STATE:
196 begin
197     if(splitIn)begin //Set of commands when the Master is send to the
198         SPLIT state.
199         case_reg                                       <= SPLITROOM_STATE;
200         masterSplitOut                                <= 1'b1;
201     end
202     else begin
203         if(preSwapIn)begin
204             data_counter                               <= data_counter + 1'b1;
205             data_shift                                 <= data_shift << 1;
206             data_shift[0]                             <= dataIn;
207             case_reg                                   <= SWAP_STATE;
208         end
209         else begin
210             if(noAddressIn)begin
211                 master_busy_state                     <= 1'b0;
212                 data_shift                             <= {
213                     DATA_SERIAL_BIT_WIDTH {1'b0} };
214                 noAddressOut                           <= 1'b1;
215                 case_reg                               <= IDLE_STATE;
216             end
217             else begin
218                 if(slaveSplitIn)begin
219                     master_busy_state                 <= 1'b0;
220                     data_shift                         <= {
221                         DATA_SERIAL_BIT_WIDTH {1'b0} };
222                     transIncompleteOut                 <= 1'b1;
223                     case_reg                           <= IDLE_STATE;
224                 end
225                 else begin
226                     case_reg                           <= JUNCTION_STATE;
227                 end
228             end
229         end
230     end
231 end
232 SPLITROOM_STATE:
233 begin
234     if(releasedIn)begin
235         case_reg                                       <= JUNCTION_STATE;
236         masterSplitOut                                <= 1'b0;
237     end
238     else begin
239         case_reg                                       <= SPLITROOM_STATE;
240         masterSplitOut                                <= 1'b1;
241     end
242 end
243 SWAP_STATE:
244 begin
245     if(data_counter < DATA_SERIAL_BIT_WIDTH + 1)begin
246         data_counter                               <= data_counter + 1'b1;
247         data_shift                                 <= data_shift << 1;
248         data_shift[0]                             <= dataIn;
249         case_reg                                   <= SWAP_STATE;

```

245	end	
246	else begin	
247	data_counter	<= { DATA_COUNTER_WIDTH
	{1'b0} };	
248	case_reg	<= COMPLETE;
249	endOut	<= 1'b1;
250	end	
251	end	
252	COMPLETE:	
253	begin	
254	case_reg	<= IDLE_STATE;
255	data_shift	<= {
	DATA_SERIAL_BIT_WIDTH {1'b0} };	
256	endOut	<= 1'b0;
257	requestOut	<= 1'b0;
258	end	
259	default:	
260	begin	
261	case_reg	<= IDLE_STATE;
262	requestOut	<= 1'b0;
263	end	
264	endcase	
265	end	
266	end	
267	endmodule	

```

1  `timescale 1ns / 1ps
2  //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
3  // Company: ENTC, University of Moratuwa
4  // Author: Miyuru Thathsara | Nadarasar Bahavan
5  //
6  // Create Date: 12/16/2020 04:51:49 PM
7  // Design Name: Address Decoder
8  // Module Name: addrDecode
9  // Project Name: System Bus
10 // Additional Comments:
11 //
12 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
13 module addrDecode(
14     clk,
15     rst_reset,
16     reset,
17     addrIn,
18     splitIn,
19     doneIn,
20     addrReadyIn,
21     addrOut,
22     addrValidOut,
23     stateRegOut,
24     ackSignalOut,
25     readyOut,
26     slaveHitOut,
27     slaveNotHitOut
28 );
29 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
30 // Parameters
31 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
32 parameter STATE_REG_WIDTH = 2;
33 parameter ADDR_SERIAL_BIT_WIDTH = 19;
34 parameter ADDR_RANGE_BEGIN = 0;
35 parameter ADDR_RANGE_END = 100;
36 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
37 // Local Parameters
38 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
39 localparam CASE_REG_WIDTH = 2;
40 localparam IDLE_STATE = 2'b00;
41 localparam CHECK_RANGE_STATE = 2'b01;
42 localparam SPLIT_STATE = 2'b10;
43 localparam ADDR_OUT_BIT_WIDTH = ADDR_SERIAL_BIT_WIDTH
44 - 1;
45 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
46 // I/O pin configuration
47 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
48 input clk;
49 input rst_reset;
50 input reset;
51 input addrIn;
52 input splitIn;
53 input doneIn;
54 input addrReadyIn;
55 output reg [ ADDR_OUT_BIT_WIDTH - 1 : 0 ] addrOut; //The
56 address of the system being given out.
57 output reg addrValidOut;
58 output reg [ STATE_REG_WIDTH - 1 : 0 ] stateRegOut;
59 output reg ackSignalOut;
60 //To the Slave and Master to initiate the next phase of transactions
61 output reg readyOut; //From
62 the Address Decoder to the Arbiter.
63 output reg slaveHitOut;
64 output reg slaveNotHitOut;
65 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
66 // Wires & Registers
67 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
68 reg [ CASE_REG_WIDTH - 1 : 0 ] case_reg;
69 reg [ ADDR_SERIAL_BIT_WIDTH - 1 : 0 ] shift_reg;

```



```

132             ackSignalOut    <= 1'b1;
133             stateRegOut      <= 2'b11;
134         end
135     else begin
136         case_reg             <= CHECK_RANGE_STATE;
137     end
138 end
139 end
140 else begin
141     case_reg                 <= IDLE_STATE;
142     slaveNotHitOut           <= 1'b1;
143     stateRegOut              <= 2'b01; //Indicates out of range and
144                                being split
145     shift_reg                <= { ADDR_SERIAL_BIT_WIDTH {1'b0} };
146     readyOut                 <= 1'b1;
147 end
148 SPLIT_STATE:
149 begin
150     slaveHitOut              <= 1'b0;
151     if(doneIn)begin
152         case_reg             <= IDLE_STATE;
153         stateRegOut           <= 2'b00; //Indicates out of range and
154                                being split
155         shift_reg             <= { ADDR_SERIAL_BIT_WIDTH {1'b0} };
156         addrOut                <= { ADDR_OUT_BIT_WIDTH {1'b0} };
157         addrValidOut           <= 1'b0;
158         readyOut              <= 1'b1;
159         ackSignalOut           <= 1'b0;
160     end
161     else begin
162         case_reg             <= SPLIT_STATE;
163     end
164 end
165 default:
166 begin
167     case_reg                 <= IDLE_STATE;
168     readyOut                 <= 1'b1;
169 end
170 endcase
171 end
172 endmodule
173

```

```

1  `timescale 1ns / 1ps
2  //////////////////////////////////////////////////
3  // Company: ENTC, University of Moratuwa
4  // Author: Miyuru Thathsara | Nadarasar Bahavan
5  //
6  // Create Date: 12/17/2020 11:34:15 PM
7  // Design Name: Slave Interface
8  // Module Name: slaveInterface
9  // Project Name: System Bus
10 // Additional Comments:
11 //
12 //////////////////////////////////////////////////
13
14
15 module slaveInterface(
16     clk,
17     rst_reset,
18     reset,
19     dataIn,
20     slaveDataIn,
21     slaveDataValidIn,
22     addrValidIn,
23     addrIn,
24     busyIn,
25     releasedIn,
26     slaveWriteResponseIn,
27     dataOut,
28     slaveDataValidOut,
29     slaveDataOut,
30     slaveAddrValidOut,
31     slaveAddrOut,
32     splitOut,
33     unsplitOut,
34     slaveReadEnOut,
35     slaveWriteEnOut,
36     slaveDataReadyOut,
37     addrReadyOut,
38     addrDoneOut,
39     preSwapOut,
40     swapGrantOut
41 );
42 `include "log.v"
43 //////////////////////////////////////////////////
44 // Parameters
45 //////////////////////////////////////////////////
46 parameter DATA_BUS_WIDTH = 16;
47 parameter ADDR_BUS_WIDTH = 16;
48 parameter ADDR_SERIAL_BIT_WIDTH = 19;
49 parameter DATA_SERIAL_BIT_WIDTH = 17;
50 //////////////////////////////////////////////////
51 // Local Parameters
52 //////////////////////////////////////////////////
53 localparam CASE_REG_BIT_WIDTH = 3;
54 localparam IDLE_STATE = 3'b000;
55 localparam JUNCTION_STATE = 3'b001;
56 localparam SPLIT_STATE = 3'b010;
57 localparam UNSPLIT_STATE = 3'b011;
58 localparam FETCH_DATA_STATE = 3'b100;
59 localparam READ_SWAP_STATE = 3'b101;
60 localparam WRITE_SWAP_STATE = 3'b110;
61 localparam WRITE_COMPLETE_STATE = 3'b111;
62 localparam READ_CASE_REG_BIT_WIDTH = 1;
63 localparam PUT_ADDR_STATE = 1'b0;
64 localparam DATA_WAIT_STATE = 1'b1;
65 localparam SWAP_COUNTER_WIDTH =
count2width(DATA_BUS_WIDTH + 1);
66 localparam ADDR_IN_BIT_WIDTH =
ADDR_SERIAL_BIT_WIDTH - 1;
67 //////////////////////////////////////////////////

```

```

68 // I/O pins
69 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
70 input                                     clk;
71 input                                     rst_reset;
72 input                                     reset;
73 input                                     dataIn;
74 input          [ DATA_BUS_WIDTH - 1 : 0 ] slaveDataIn;
75 input                                     slaveDataValidIn;
76 input                                     addrValidIn;
77 input          [ ADDR_IN_BIT_WIDTH - 1 : 0 ] addrIn;
78 input                                     busyIn;
79 input                                     releasedIn;
80 input                                     slaveWriteResponseIn;
81 output                                     dataOut;
82 output reg                               slaveDataValidOut;
83 output reg          [ DATA_BUS_WIDTH - 1 : 0 ] slaveDataOut;
84 output reg                               slaveAddrValidOut;
85 output reg          [ ADDR_BUS_WIDTH - 1 : 0 ] slaveAddrOut;
86 output reg                               splitOut;
87 output reg                               unsplitOut;
88 output reg                               slaveReadEnOut;
89 output reg                               slaveWriteEnOut;
90 output reg                               slaveDataReadyOut;
91 output reg                               addrReadyOut;
92 output reg                               addrDoneOut;
93 output reg                               preSwapOut;
94 output reg                               swapGrantOut;
95 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
96 // Wires and Registers
97 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
98 reg          [ CASE_REG_BIT_WIDTH - 1 : 0 ] case_reg;
99 reg          [ READ_CASE_REG_BIT_WIDTH - 1 : 0 ] read_case_reg;
100 reg          [ DATA_SERIAL_BIT_WIDTH - 1 : 0 ] data_reg;
101 reg          [ SWAP_COUNTER_WIDTH - 1 : 0 ] swap_counter;
102 reg                               swap_counter_flag;
103 reg                               reset_a;
104 reg                               reset_b;
105 reg          [ ADDR_IN_BIT_WIDTH - 1 : 0 ] addr_integrate_reg;
106 wire                               reset_wire;
107 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
108 // Implementation
109 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
110
111 assign reset_wire = reset | reset_b;
112 assign dataOut = data_reg[
DATA_SERIAL_BIT_WIDTH - 1 ];
113
114 always@(posedge clk)begin
115     if(~rst_reset)begin
116         reset_a <= 1'b0;
117         reset_b <= 1'b0;
118     end
119     else begin
120         reset_a <= reset;
121         reset_b <= reset_a;
122     end
123 end
124
125 always@(posedge clk or negedge reset_wire)begin
126     if(~reset_wire)begin
127         slaveDataValidOut <= 1'b0;
128         slaveDataOut <= { DATA_BUS_WIDTH
{1'b0} };
129         slaveAddrValidOut <= 1'b0;
130         slaveAddrOut <= { ADDR_BUS_WIDTH
{1'b0} };
131         splitOut <= 1'b0;
132         unsplitOut <= 1'b0;
133         slaveDataReadyOut <= 1'b0;

```

```

134         addrReadyOut                                     <= 1'b0;
135         slaveReadEnOut                                   <= 1'b0;
136         slaveWriteEnOut                                 <= 1'b0;
137         data_reg                                         <= {
138             DATA_SERIAL_BIT_WIDTH {1'b0} };
139         addr_integrate_reg                                <= {
140             ADDR_IN_BIT_WIDTH {1'b0} };
141         read_case_reg                                     <= 1'b0;
142         case_reg                                          <= {
143             CASE_REG_BIT_WIDTH {1'b0} };
144         swap_counter                                      <= {
145             SWAP_COUNTER_WIDTH {1'b0} };
146         swap_counter_flag                                 <= 1'b0;
147         addrDoneOut                                       <= 1'b0;
148         preSwapOut                                       <= 1'b0;
149         swapGrantOut                                     <= 1'b0;
150     end
151     else begin
152         case(case_reg)
153             IDLE_STATE:
154                 begin
155                     if(addrReadyOut)begin
156                         if(addrValidIn)begin
157                             addr_integrate_reg          <= addrIn;
158                             addrReadyOut                 <= 1'b0;
159                             addrDoneOut                 <= 1'b1;
160                             case_reg                   <= JUNCTION_STATE;
161                         end
162                     else begin
163                         addrReadyOut                     <= 1'b1;
164                         addrDoneOut                     <= 1'b0;
165                         case_reg                         <= IDLE_STATE;
166                     end
167                 end
168             end
169             JUNCTION_STATE:
170                 begin
171                     unsplitOut                            <= 1'b0;
172                     if(busyIn)begin
173                         swapGrantOut                     <= 1'b0;
174                         splitOut                         <= 1'b1;
175                         case_reg                         <= SPLIT_STATE;
176                     end
177                     else begin
178                         splitOut                          <= 1'b0;
179                         if(addr_integrate_reg[ ADDR_IN_BIT_WIDTH - 1 ])begin
180                             swapGrantOut                 <= 1'b1;
181                             case_reg                   <= FETCH_DATA_STATE;
182                         end
183                     else begin
184                         if(addr_integrate_reg[ADDR_IN_BIT_WIDTH - 2])begin
185                             swapGrantOut                 <= 1'b1;
186                             case_reg                   <= WRITE_SWAP_STATE;
187                         end
188                     else begin
189                         swapGrantOut                     <= 1'b0;
190                         addrReadyOut                     <= 1'b1;
191                         addrDoneOut                     <= 1'b0;
192                         case_reg                         <= IDLE_STATE;
193                         addr_integrate_reg               <= {
194                             ADDR_IN_BIT_WIDTH {1'b0} };
195                     end
196                 end
197             end

```

```

198     end
199     SPLIT_STATE:
200     begin
201         splitOut                                     <= 1'b1;
202         if(~busyIn)begin
203             case_reg                                <= UNSPLIT_STATE;
204         end
205         else begin
206             case_reg                                <= SPLIT_STATE;
207         end
208     end
209     UNSPLIT_STATE:
210     begin
211         if(releasedIn)begin
212             unsplitOut                               <= 1'b0;
213             splitOut                                 <= 1'b0;
214             case_reg                                <= JUNCTION_STATE;
215         end
216         else begin
217             unsplitOut                               <= 1'b1;
218             splitOut                                 <= 1'b1;
219             case_reg                                <= UNSPLIT_STATE;
220         end
221     end
222     FETCH_DATA_STATE:
223     begin
224         case(read_case_reg)
225             PUT_ADDR_STATE:
226             begin
227                 slaveReadEnOut                       <= 1'b1;
228                 slaveDataReadyOut                     <= 1'b1;
229                 slaveAddrOut                           <=
230                 addr_integrate_reg[ ADDR_IN_BIT_WIDTH - 3 : 0 ];
231                 slaveAddrValidOut                     <= 1'b1;
232                 read_case_reg                         <= DATA_WAIT_STATE;
233                 case_reg                              <= FETCH_DATA_STATE;
234             end
235             DATA_WAIT_STATE:
236             begin
237                 if(slaveDataValidIn)begin
238                     preSwapOut                         <= 1'b1;
239                     slaveReadEnOut                     <= 1'b0;
240                     slaveDataReadyOut                 <= 1'b0;
241                     slaveAddrValidOut                 <= 1'b0;
242                     slaveAddrOut                       <= { ADDR_BUS_WIDTH
243                     {1'b0} };
244                     data_reg                           <= { 1'b1,
245                     slaveDataIn };
246                     read_case_reg                     <= PUT_ADDR_STATE;
247                     case_reg                           <= READ_SWAP_STATE;
248                     swap_counter                       <= {
249                     SWAP_COUNTER_WIDTH {1'b0} };
250                 end
251                 else begin
252                     read_case_reg                     <= DATA_WAIT_STATE;
253                     case_reg                           <= FETCH_DATA_STATE;
254                 end
255             end
256         endcase
257     end
258     READ_SWAP_STATE:
259     begin
260         if(swap_counter == 0)begin
261             swap_counter                               <= swap_counter +
262             1'b1;
263             preSwapOut                               <= 1'b1;
264             case_reg                                  <= READ_SWAP_STATE;
265         end
266         else begin

```

```

262         if(swap_counter < DATA_SERIAL_BIT_WIDTH)begin
263             swap_counter
264                 1'b1;
265             data_reg
266                 1'b1;
267             preSwapOut
268                 1'b1;
269             case_reg
270                 READ_SWAP_STATE;
271         end
272     else begin
273         swapGrantOut
274             1'b0;
275         addr_integrate_reg
276             {
277                 ADDR_IN_BIT_WIDTH {1'b0} };
278         addrReadyOut
279             1'b1;
280         addrDoneOut
281             1'b0;
282         preSwapOut
283             1'b0;
284         data_reg
285             {
286                 DATA_SERIAL_BIT_WIDTH {1'b0} };
287         swap_counter
288             {
289                 SWAP_COUNTER_WIDTH {1'b0} };
290         case_reg
291             IDLE_STATE;
292     end
293 end
294
295 WRITE_SWAP_STATE:
296 begin
297     if(swap_counter_flag)begin
298         if(swap_counter < DATA_SERIAL_BIT_WIDTH)begin
299             swap_counter
300                 1'b1;
301             data_reg
302                 1'b1;
303             preSwapOut
304                 1'b1;
305             data_reg[0]
306                 dataIn;
307             case_reg
308                 WRITE_SWAP_STATE;
309         end
310     else begin
311         swapGrantOut
312             1'b0;
313         swap_counter_flag
314             1'b0;
315         swap_counter
316             {
317                 SWAP_COUNTER_WIDTH {1'b0} };
318         slaveWriteEnOut
319             1'b1;
320         slaveDataOut
321             data_reg[
322                 DATA_SERIAL_BIT_WIDTH - 2 : 0 ];
323         slaveAddrOut
324             {
325                 addr_integrate_reg[ ADDR_IN_BIT_WIDTH - 3 : 0 ];
326                 slaveDataValidOut
327                 1'b1;
328                 slaveAddrValidOut
329                 1'b1;
330                 preSwapOut
331                 1'b0;
332                 case_reg
333                 WRITE_COMPLETE_STATE;
334             }
335         end
336     else begin
337         if(dataIn)begin
338             swap_counter_flag
339                 1'b1;
340             swap_counter
341                 {
342                     SWAP_COUNTER_WIDTH {1'b0} };
343             preSwapOut
344                 1'b1;
345         end
346         case_reg
347             WRITE_SWAP_STATE;
348     end
349 end
350
351 WRITE_COMPLETE_STATE:
352 begin
353     if(slaveWriteResponseIn)begin
354         addr_integrate_reg
355             {
356                 ADDR_IN_BIT_WIDTH {1'b0} };
357         slaveWriteEnOut
358             1'b0;
359         slaveDataOut
360             {
361                 DATA_BUS_WIDTH
362                 {1'b0} };
363         slaveAddrOut
364             {
365                 ADDR_BUS_WIDTH

```

```

319         {1'b0} };
320         slaveDataValidOut
321         slaveAddrValidOut
322         addrReadyOut
323         addrDoneOut
324         data_reg
325         DATA_SERIAL_BIT_WIDTH {1'b0} };
326         case_reg
327         case_reg
328         WRITE_COMPLETE_STATE;
329         end
330         endcase
331         end
332         end
333         endmodule

```

```

<= 1'b0;
<= 1'b0;
<= 1'b1;
<= 1'b0;
<= {
<= IDLE_STATE;
<=

```

SELF CHECKING TESTBENCH

```

1  `timescale 1ns / 1ps
2  //////////////////////////////////////////////////
3  // Company: ENTC, University of Moratuwa
4  // Author: Miyuru Thathsara | Nadarasar Bahavan
5  //
6  // Create Date: 12/23/2020 08:17:03 AM
7  // Design Name: Top module testbench
8  // Module Name: top_TB
9  // Project Name: System Bus
10 // Additional Comments:
11 //
12 //////////////////////////////////////////////////
13
14
15 module top_TB();
16
17 //////////////////////////////////////////////////
18 // Parameters
19 //////////////////////////////////////////////////
20 parameter MASTER_INT_STATE_REG_WIDTH           = 3;
21 parameter ADDR_DECODE_STATE_REG_WIDTH           = 2;
22 parameter NUM_OF_MASTERS                        = 2;
23 parameter NUM_OF_SLAVES                        = 3;
24 parameter DATA_BUS_WIDTH                      = 16;
25 parameter ADDR_BUS_WIDTH                      = 16;
26 parameter ADDR_SERIAL_BIT_WIDTH                = 19;
27 parameter DATA_SERIAL_BIT_WIDTH                = 17;
28 parameter SLAVE1_ADDR_RANGE_BEGIN              = 0;
29 parameter SLAVE1_ADDR_RANGE_END                = 1024;
30 parameter SLAVE2_ADDR_RANGE_BEGIN              = 1024;
31 parameter SLAVE2_ADDR_RANGE_END                = 2048;
32 parameter SLAVE3_ADDR_RANGE_BEGIN              = 2048;
33 parameter SLAVE3_ADDR_RANGE_END                = 4096;
34 //////////////////////////////////////////////////
35 // Local Parameters
36 //////////////////////////////////////////////////
37 localparam NUM_OF_TESTS                        = 10;
38 localparam DATA_BOUND                        = 1000;
39 localparam ADDR_BOUND                        = 4096;
40 //////////////////////////////////////////////////
41 // Wires and Registers
42 //////////////////////////////////////////////////
43 reg                                             clk;
44 reg                                             rst_reset;
45 reg                                             reset;
46
47 boss1RequestIn;
48 reg                                             boss2RequestIn;
49 reg [ ADDR_BUS_WIDTH - 1 : 0 ] boss1AddressIn;
50 reg [ ADDR_BUS_WIDTH - 1 : 0 ] boss2AddressIn;
51 reg boss1AddressValidIn;
52 reg boss2AddressValidIn;
53 reg boss1ReadIn;
54 reg boss2ReadIn;
55 reg boss1WriteIn;
56 reg [ DATA_BUS_WIDTH - 1 : 0 ] boss2WriteIn;
57 reg [ DATA_BUS_WIDTH - 1 : 0 ] boss1DataIn;
58 reg [ DATA_BUS_WIDTH - 1 : 0 ] boss2DataIn;
59 boss1DataValidIn;
60 reg boss2DataValidIn;
61 reg [ DATA_BUS_WIDTH - 1 : 0 ] slave1DataIn;
62 reg [ DATA_BUS_WIDTH - 1 : 0 ] slave2DataIn;

```

```

62     reg                                [ DATA_BUS_WIDTH - 1 : 0 ]
63     reg                                slave3DataIn;
64     reg                                slave1DataValidIn;
65     reg                                slave2DataValidIn;
66     reg                                slave3DataValidIn;
67     reg                                slave1WriteResponseIn;
68     reg                                slave2WriteResponseIn;
69     reg                                slave3WriteResponseIn;
70     reg                                slave1BusyIn;
71     reg                                slave2BusyIn;
72     wire                                slave3BusyIn;
73     wire                                boss1DataOut;
74     wire                                boss2DataOut;
75     wire                                boss1DataValidOut;
76     wire                                boss2DataValidOut;
77     wire                                boss1NoAddressOut;
78     wire                                boss2NoAddressOut;
79     wire                                slave1DataValidOut;
80     wire                                slave2DataValidOut;
81     wire                                slave3DataValidOut;
82     wire                                slave1DataOut;
83     wire                                slave2DataOut;
84     wire                                slave3DataOut;
85     wire                                slave1AddrValidOut;
86     wire                                slave2AddrValidOut;
87     wire                                slave3AddrValidOut;
88     wire                                slave1AddrOut;
89     wire                                slave2AddrOut;
90     wire                                slave3AddrOut;
91     wire                                slave1ReadEnOut;
92     wire                                slave2ReadEnOut;
93     wire                                slave3ReadEnOut;
94     wire                                slave1WriteEnOut;
95     wire                                slave2WriteEnOut;
96     wire                                slave3WriteEnOut;
97     wire                                slave1DataReadyOut;
98     wire                                slave2DataReadyOut;
99     wire                                slave3DataReadyOut;

```

```

100 boss1DataAddrReadyOut;
101 wire
102 boss2DataAddrReadyOut;
103 wire
104 boss1TransIncompleteOut;
105 wire
106 boss2TransIncompleteOut;
107 integer i1;
108 // Implementation
109 ///////////////////////////////////////////////////////////////////
110 // Implementation
111 ///////////////////////////////////////////////////////////////////
112 // Bounded Random Number generator function
113 function [ ADDR_BUS_WIDTH - 1 : 0 ] randomBoundNum;
114     input integer bound;
115     begin
116         randomBoundNum = $urandom % bound;
117     end
118 endfunction
119
120 always #5 clk                                     <= ~clk;
121
122 initial begin
123     #0;
124     clk                                     = 1'b0;
125     rst_reset                             = 1'b0;
126     reset                                 = 1'b0;
127     // boss communicating ports
128     boss1RequestIn                       = 1'b0;
129     boss2RequestIn                       = 1'b0;
130     boss1AddressIn                       = {
131     ADDR_BUS_WIDTH {1'b0} };
132     boss2AddressIn                       = {
133     ADDR_BUS_WIDTH {1'b0} };
134     boss1AddressValidIn                  = 1'b0;
135     boss2AddressValidIn                  = 1'b0;
136     boss1ReadIn                          = 1'b0;
137     boss2ReadIn                          = 1'b0;
138     boss1WriteIn                         = 1'b0;
139     boss2WriteIn                         = 1'b0;
140     boss1DataIn                          = {
141     DATA_BUS_WIDTH {1'b0} };
142     boss2DataIn                          = {
143     DATA_BUS_WIDTH {1'b0} };
144     boss1DataValidIn                     = 1'b0;
145     boss2DataValidIn                     = 1'b0;
146     // slave communicating ports
147     slavel1DataIn                        = {
148     DATA_BUS_WIDTH {1'b0} };
149     slave2DataIn                         = {
150     DATA_BUS_WIDTH {1'b0} };
151     slave3DataIn                         = {
152     DATA_BUS_WIDTH {1'b0} };
153     slavel1DataValidIn                   = 1'b0;
154     slave2DataValidIn                   = 1'b0;
155     slave3DataValidIn                   = 1'b0;
156     slavel1WriteResponseIn               = 1'b0;
157     slave2WriteResponseIn               = 1'b0;
158     slave3WriteResponseIn               = 1'b0;
159     slavel1BusyIn                        = 1'b0;
160     slave2BusyIn                        = 1'b0;
161     slave3BusyIn                        = 1'b0;
162     #20;
163     rst_reset                             = 1'b1;
164     #20;
165     rst_reset                             = 1'b0;
166     #20;
167     rst_reset                             = 1'b1;
168     reset                                 = 1'b1;

```

```

158 #20;
159 reset = 1'b0;
160 #20;
161 reset = 1'b1;
162 // for(i1 = 0; i1 < NUM_OF_TESTS; i1 = i1 + 1)begin
163 #10;
164
165 // Random Read and Write block
166
167 // boss1RequestIn =
168 randomBoundNum(2);
169 // if(boss1RequestIn)begin
170 // boss2RequestIn =
171 randomBoundNum(2);
172 // end
173 // else begin
174 // boss2RequestIn =
175 1'b1;
176 // end
177 // case({boss1RequestIn, boss2RequestIn})
178 // 2'b00:
179 // begin
180 // boss1AddressIn = {
181 ADDR_BUS_WIDTH {1'b0} };
182 // boss2AddressIn = {
183 ADDR_BUS_WIDTH {1'b0} };
184 // boss1AddressValidIn =
185 1'b0;
186 // boss2AddressValidIn =
187 1'b0;
188 // boss1ReadIn =
189 1'b0;
190 // boss2ReadIn =
191 1'b0;
192 // boss1WriteIn =
193 1'b0;
194 // boss2WriteIn =
195 1'b0;
196 // boss1DataIn = {
197 DATA_BUS_WIDTH {1'b0} };
198 // boss2DataIn = {
199 DATA_BUS_WIDTH {1'b0} };
200 // boss1DataValidIn =
201 1'b0;
202 // boss2DataValidIn =
203 1'b0;
204 // end
205 // 2'b01:
206 // begin
207 // boss1AddressIn = {
208 ADDR_BUS_WIDTH {1'b0} };
209 // boss2AddressIn =
210 randomBoundNum(ADDR_BOUND);
211 // boss1AddressValidIn =
212 1'b0;
213 // boss2AddressValidIn =
214 1'b1;
215 // boss1ReadIn =
216 1'b0;
217 // boss2ReadIn =
218 randomBoundNum(2);
219 // boss1WriteIn =
220 1'b0;
221 // boss2WriteIn =
222 ~boss2ReadIn;

```

```

200          //          boss1DataIn                                = {
DATA_BUS_WIDTH {1'b0} };
201          //          boss1DataValidIn                            =
1'b0;
202          //          if(boss2WriteIn)begin
203          //          boss2DataIn                                    =
randomBoundNum(DATA_BOUND);
204          //          boss2DataValidIn                            =
1'b1;
205          //          end
206          //          else begin
207          //          boss2DataIn                                    = {
DATA_BUS_WIDTH {1'b0} };
208          //          boss2DataValidIn                            =
1'b0;
209          //          end
210          //          end
211          //          2'b10:
212          //          begin
213          //          boss1AddressIn                                =
randomBoundNum(ADDR_BOUND);
214          //          boss2AddressIn                                = {
ADDR_BUS_WIDTH {1'b0} };
215          //          boss1AddressValidIn                          =
1'b1;
216          //          boss2AddressValidIn                          =
1'b0;
217          //          boss1ReadIn                                  =
randomBoundNum(2);
218          //          boss2ReadIn                                  =
1'b0;
219          //          boss1WriteIn                                  =
~boss1ReadIn;
220          //          boss2WriteIn                                  =
1'b0;
221          //          boss2DataIn                                    = {
DATA_BUS_WIDTH {1'b0} };
222          //          boss2DataValidIn                            =
1'b0;
223          //          if(boss1WriteIn)begin
224          //          boss1DataIn                                    =
randomBoundNum(DATA_BOUND);
225          //          boss1DataValidIn                            =
1'b1;
226          //          end
227          //          else begin
228          //          boss1DataIn                                    = {
DATA_BUS_WIDTH {1'b0} };
229          //          boss1DataValidIn                            =
1'b0;
230          //          end
231          //          end
232          //          2'b11:
233          //          begin
234          //          boss1AddressIn                                =
randomBoundNum(ADDR_BOUND);
235          //          boss2AddressIn                                =
randomBoundNum(ADDR_BOUND);
236          //          boss1AddressValidIn                          =
1'b1;
237          //          boss2AddressValidIn                          =
1'b1;
238          //          boss1ReadIn                                  =
randomBoundNum(2);
239          //          boss2ReadIn                                  =
randomBoundNum(2);
240          //          boss1WriteIn                                  =
~boss1ReadIn;
241          //          boss2WriteIn                                  =

```

```

~boss2ReadIn;
242 //          if(boss1WriteIn)begin
243 //              boss1DataIn                                     =
randomBoundNum(DATA_BOUND);
244 //              boss1DataValidIn                               =
1'b1;
245 //          end
246 //          else begin
247 //              boss1DataIn                                     = {
DATA_BUS_WIDTH {1'b0} };
248 //              boss1DataValidIn                               =
1'b0;
249 //          end
250 //          if(boss2WriteIn)begin
251 //              boss2DataIn                                     =
randomBoundNum(DATA_BOUND);
252 //              boss2DataValidIn                               =
1'b1;
253 //          end
254 //          else begin
255 //              boss2DataIn                                     = {
DATA_BUS_WIDTH {1'b0} };
256 //              boss2DataValidIn                               =
1'b0;
257 //          end
258 //      end
259 // endcase
260
////////////////////////////////////
////////////////////////////////////
261 boss1RequestIn                                     = 1'b1;
262 boss2RequestIn                                     = 1'b1;
263 boss1AddressIn                                     = 1055;
264 boss2AddressIn                                     = 1300;
265 boss1AddressValidIn                               = 1'b1;
266 boss2AddressValidIn                               = 1'b1;
267 boss1ReadIn                                        = 1'b1;
268 boss2ReadIn                                        = 1'b1;
269 boss1WriteIn                                       = 1'b0;
270 boss2WriteIn                                       = 1'b0;
271 boss1DataIn                                         = {
DATA_BUS_WIDTH {1'b0} };
272 boss2DataIn                                         = {
DATA_BUS_WIDTH {1'b0} };
273 boss1DataValidIn                                   = 1'b0;
274 boss2DataValidIn                                   = 1'b0;
275 $display("Master1 Addressing Address: ", boss1AddressIn);
276 $display("Master2 Addressing Address: ", boss2AddressIn);
277 if(boss1AddressIn >= SLAVE1_ADDR_RANGE_BEGIN && boss1AddressIn <
SLAVE1_ADDR_RANGE_END)begin
278     $display("Master1 wants to communicate with slave1!!!!");
279 end
280 else begin
281     if(boss1AddressIn >= SLAVE2_ADDR_RANGE_BEGIN && boss1AddressIn <
SLAVE2_ADDR_RANGE_END)begin
282         $display("Master1 wants to communicate with slave2!!!!");
283     end
284     else begin
285         if(boss1AddressIn >= SLAVE3_ADDR_RANGE_BEGIN && boss1AddressIn <
SLAVE3_ADDR_RANGE_END)begin
286             $display("Master1 wants to communicate with slave3!!!!");
287         end
288         else begin
289             $display("Address is out of space!!!!");
290         end
291     end
292 end
293 if(boss2AddressIn >= SLAVE1_ADDR_RANGE_BEGIN && boss2AddressIn <
SLAVE1_ADDR_RANGE_END)begin

```

```

294         $display("Master2 wants to communicate with slave1!!!!");
295     end
296     else begin
297         if(boss2AddressIn >= SLAVE2_ADDR_RANGE_BEGIN && boss2AddressIn <
           SLAVE2_ADDR_RANGE_END)begin
298             $display("Master2 wants to communicate with slave2!!!!");
299         end
300         else begin
301             if(boss2AddressIn >= SLAVE3_ADDR_RANGE_BEGIN && boss2AddressIn <
           SLAVE3_ADDR_RANGE_END)begin
302                 $display("Master2 wants to communicate with slave3!!!!");
303             end
304             else begin
305                 $display("Address is out of space!!!!");
306             end
307         end
308     end
309     // Wait till master interface is ready to receive data and address from master
310     while(~boss1DataAddrReadyOut && ~boss2DataAddrReadyOut)begin
311         @(posedge clk);
312     end
313
314     boss1RequestIn                                     =
315     1'b0;
316     boss2RequestIn                                     =
317     1'b0;
318     boss1AddressValidIn                               =
319     1'b0;
320     boss2AddressValidIn                               =
321     1'b0;
322     boss1DataValidIn                                  =
323     1'b0;
324     boss2DataValidIn                                  =
325     1'b0;
326
327     // if(boss1RequestIn)begin
328     //     boss1RequestIn
329     = 1'b0;
330     //     boss1AddressValidIn
331     = 1'b0;
332     //     boss1DataValidIn
333     = 1'b0;
334     // end
335     // else begin
336     //     boss2RequestIn
337     = 1'b0;
338     //     boss2AddressValidIn
339     = 1'b0;
340     //     boss2DataValidIn
341     = 1'b0;
342     // end
343
344     // slave1BusyIn
345     = 1'b1;
346     slave2BusyIn                                     =
347     1'b1;
348     // slave3BusyIn
349     = 1'b1;
350     #300;
351     // slave1BusyIn
352     = 1'b0;
353     slave2BusyIn                                     =
354     1'b0;
355     // slave3BusyIn
356     = 1'b0;
357
358     //////////////////////////////////////
359     //////////////////////////////////////

```

```

341 // Random Read and Write block
342
343 // if(boss1DataAddrReadyOut)begin
344 //     $display("Master1 has granted Permission by the Arbitor!!!!");
345 //     if(boss1ReadIn)begin
346 //         // Read data operation
347 //         while(~slavel1AddrValidOut && ~slave2AddrValidOut &&
~slave3AddrValidOut)begin
348 //             @(posedge clk);
349 //         end
350
351 //         if(slavel1AddrValidOut)begin
352 //             slavel1DataIn
=
randomBoundNum(DATA_BOUND);
353 //             slavel1DataValidIn
=
1'b1;
354 //         end
355 //         else begin
356 //             if(slave2AddrValidOut)begin
357 //                 slave2DataIn
=
randomBoundNum(DATA_BOUND);
358 //                 slave2DataValidIn
= 1'b1;
359 //             end
360 //             else begin
361 //                 if(slave3AddrValidOut)begin
362 //                     slave3DataIn
=
randomBoundNum(DATA_BOUND);
363 //                     slave3DataValidIn
= 1'b1;
364 //                 end
365 //             end
366 //         end
367
368 //         while(slavel1AddrValidOut || slave2AddrValidOut ||
slave3AddrValidOut)begin
369 //             @(posedge clk);
370 //         end
371
372 //         slavel1DataIn
= { DATA_BUS_WIDTH {1'b0} };
373 //         slave2DataIn
= { DATA_BUS_WIDTH {1'b0} };
374 //         slave3DataIn
= { DATA_BUS_WIDTH {1'b0} };
375 //         slavel1DataValidIn
= 1'b0;
376 //         slave2DataValidIn
= 1'b0;
377 //         slave3DataValidIn
= 1'b0;
378 //     end
379 //     else begin
380 //         if(boss1WriteIn)begin
381 //             // Write data operation
382 //             // Wait till Adress valid or data valid comes out to the slave
383 //             while(~slavel1AddrValidOut && ~slavel1DataValidOut &&

```

```

~slave2AddrValidOut && ~slave2DataValidOut && ~slave2AddrValidOut &&
~slave3DataValidOut)begin
384 //          @(posedge clk);
385 //          end
386
387 //          if(slave1AddrValidOut || slave1DataValidOut)begin
388 //
389 slave1WriteResponseIn                                = 1'b1;
390 //          end
391 //          else begin
392 //          if(slave2AddrValidOut || slave2DataValidOut)begin
393 //
394 slave2WriteResponseIn                                = 1'b1;
395 //          end
396 //          else begin
397 //          if(slave3AddrValidOut || slave3DataValidOut)begin
398 //
399 slave3WriteResponseIn                                = 1'b1;
400 //          end
401 //          else begin
402 //          $display("Something Wrong!!!!");
403 //          end
404 //          end
405 //          end
406 //          #30;
407 //          end
408 //          end
409 //          end
410 //          else begin
411 //          $display("Master2 has granted Permission by the Arbitor!!!!");
412 //          if(boss2ReadIn)begin
413 //          // Read data operation
414 //          while(~slave1AddrValidOut && ~slave2AddrValidOut &&
~slave3AddrValidOut)begin
415 //          @(posedge clk);
416 //          end
417
418 //          if(slave1AddrValidOut)begin
419 //
420 slave1DataIn                                =
randomBoundNum(DATA_BOUND);
421 //
422 slave1DataValidIn                                =
1'b1;
423 //          end
424 //          else begin
425 //          if(slave2AddrValidOut)begin
426 //
427 slave2DataIn                                =
randomBoundNum(DATA_BOUND);
428 //
429 slave2DataValidIn                                = 1'b1;
430 //          end
431 //          else begin
432 //          if(slave3AddrValidOut)begin
433 //
434 slave3DataIn                                =
randomBoundNum(DATA_BOUND);
435 //
436 slave3DataValidIn                                = 1'b1;

```

```

431         //             end
432         //             end
433         //             end
434
435         //             while(slave1AddrValidOut || slave2AddrValidOut ||
slave3AddrValidOut)begin
436         //                 @(posedge clk);
437         //                 end
438
439         //
slave1DataIn
= { DATA_BUS_WIDTH {1'b0} };
440         //
slave2DataIn
= { DATA_BUS_WIDTH {1'b0} };
441         //
slave3DataIn
= { DATA_BUS_WIDTH {1'b0} };
442         //
slave1DataValidIn
= 1'b0;
443         //
slave2DataValidIn
= 1'b0;
444         //
slave3DataValidIn
= 1'b0;
445         //             end
446         //             else begin
447         //                 if(boss2WriteIn)begin
448         //                     // Write data operation
449         //                     // Wait till Adress valid or data valid comes out to the slave
450         //                     while(~slave1AddrValidOut && ~slave1DataValidOut &&
~slave2AddrValidOut && ~slave2DataValidOut && ~slave2AddrValidOut &&
~slave3DataValidOut)begin
451         //                         @(posedge clk);
452         //                         end
453
454         //                     if(slave1AddrValidOut || slave1DataValidOut)begin
455         //
slave1WriteResponseIn                                     = 1'b1;
456         //                     end
457         //                     else begin
458         //                         if(slave2AddrValidOut || slave2DataValidOut)begin
459         //
slave2WriteResponseIn                                     = 1'b1;
460         //                         end
461         //                         else begin
462         //                             if(slave3AddrValidOut || slave3DataValidOut)begin
463         //
slave3WriteResponseIn                                     = 1'b1;
464         //                             end
465         //                             else begin
466         //                                 $display("Something Wrong!!!!");
467         //                             end
468         //                         end
469         //                     end
470         //                     #30;
471         //
slave1WriteResponseIn                                     =
1'b0;
472         //
slave2WriteResponseIn                                     =
1'b0;
473         //
slave3WriteResponseIn                                     =
1'b0;
474         //             end
475         //             end

```

```

476 // end
477
478 ///////////////////////////////////////////////////
479 // Write data operation
480 // Wait till Adress valid or data valid comes out to the slave
481 // while(~slavel1AddrValidOut && ~slavel1DataValidOut && ~slave2AddrValidOut &&
482 // ~slave2DataValidOut && ~slave2AddrValidOut && ~slave3DataValidOut)begin
483 //     @(posedge clk);
484 // end
485
486 // if(slavel1AddrValidOut || slavel1DataValidOut)begin
487 //     slavel1WriteResponseIn
488 //         = 1'b1;
489 // end
490 // else begin
491 //     if(slave2AddrValidOut || slave2DataValidOut)begin
492 //         slave2WriteResponseIn
493 //             = 1'b1;
494 //     end
495 //     else begin
496 //         if(slave3AddrValidOut || slave3DataValidOut)begin
497 //             slave3WriteResponseIn
498 //                 = 1'b1;
499 //         end
500 //         else begin
501 //             $display("Something Wrong!!!!");
502 //         end
503 //     end
504 // end
505 // #30;
506 // slavel1WriteResponseIn
507 //     = 1'b0;
508 // slave2WriteResponseIn
509 //     = 1'b0;
510 // slave3WriteResponseIn
511 //     = 1'b0;
512
513 // while(~slavel1AddrValidOut && ~slavel1DataValidOut && ~slave2AddrValidOut &&
514 // ~slave2DataValidOut && ~slave2AddrValidOut && ~slave3DataValidOut)begin
515 //     @(posedge clk);
516 // end
517
518 // if(slavel1AddrValidOut || slavel1DataValidOut)begin
519 //     slavel1WriteResponseIn
520 //         = 1'b1;
521 // end
522 // else begin
523 //     if(slave2AddrValidOut || slave2DataValidOut)begin
524 //         slave2WriteResponseIn
525 //             = 1'b1;
526 //     end
527 //     else begin
528 //         if(slave3AddrValidOut || slave3DataValidOut)begin
529 //             slave3WriteResponseIn
530 //                 = 1'b1;
531 //         end
532 //         else begin
533 //             $display("Something Wrong!!!!");
534 //         end
535 //     end
536 // end
537 // #30;
538 // slavel1WriteResponseIn
539 //     = 1'b0;
540 // slave2WriteResponseIn
541 //     = 1'b0;
542 // slave3WriteResponseIn

```

```

530     = 1'b0;
531     // Read data operation
532     while(~slave1AddrValidOut && ~slave2AddrValidOut && ~slave3AddrValidOut)begin
533         @(posedge clk);
534     end
535
536     if(slave1AddrValidOut)begin
537         slave1DataIn
538             = randomBoundNum(DATA_BOUND);
539         slave1DataValidIn
540             = 1'b1;
541     end
542     else begin
543         if(slave2AddrValidOut)begin
544             slave2DataIn
545                 = randomBoundNum(DATA_BOUND);
546             slave2DataValidIn
547                 = 1'b1;
548         end
549         else begin
550             if(slave3AddrValidOut)begin
551                 slave3DataIn
552                     = randomBoundNum(DATA_BOUND);
553                 slave3DataValidIn
554                     = 1'b1;
555             end
556         end
557     end
558
559     while(slave1AddrValidOut || slave2AddrValidOut || slave3AddrValidOut)begin
560         @(posedge clk);
561     end
562
563     slave1DataIn
564     = { DATA_BUS_WIDTH {1'b0} };
565     slave2DataIn
566     = { DATA_BUS_WIDTH {1'b0} };
567     slave3DataIn
568     = { DATA_BUS_WIDTH {1'b0} };
569     slave1DataValidIn
570     = 1'b0;
571     slave2DataValidIn
572     = 1'b0;
573     slave3DataValidIn
574     = 1'b0;
575
576     while(~slave1AddrValidOut && ~slave2AddrValidOut && ~slave3AddrValidOut)begin
577         @(posedge clk);
578     end
579
580     if(slave1AddrValidOut)begin
581         slave1DataIn
582             = randomBoundNum(DATA_BOUND);
583         slave1DataValidIn
584             = 1'b1;
585     end
586     else begin
587         if(slave2AddrValidOut)begin
588             slave2DataIn
589                 = randomBoundNum(DATA_BOUND);
590             slave2DataValidIn
591                 = 1'b1;
592         end
593         else begin
594             if(slave3AddrValidOut)begin
595                 slave3DataIn
596                     = randomBoundNum(DATA_BOUND);
597                 slave3DataValidIn

```

```

581         = 1'b1;
582     end
583 end
584
585 while(slave1AddrValidOut || slave2AddrValidOut || slave3AddrValidOut)begin
586     @(posedge clk);
587 end
588
589     slave1DataIn
590     = { DATA_BUS_WIDTH {1'b0} };
591     slave2DataIn
592     = { DATA_BUS_WIDTH {1'b0} };
593     slave3DataIn
594     = { DATA_BUS_WIDTH {1'b0} };
595     slave1DataValidIn
596     = 1'b0;
597     slave2DataValidIn
598     = 1'b0;
599     slave3DataValidIn
600     = 1'b0;
601
602 // end
603 end
604
605 top#(
606     .MASTER_INT_STATE_REG_WIDTH(MASTER_INT_STATE_REG_WIDTH),
607     .ADDR_DECODE_STATE_REG_WIDTH(ADDR_DECODE_STATE_REG_WIDTH),
608     .NUM_OF_MASTERS(NUM_OF_MASTERS),
609     .NUM_OF_SLAVES(NUM_OF_SLAVES),
610     .DATA_BUS_WIDTH(DATA_BUS_WIDTH),
611     .ADDR_BUS_WIDTH(ADDR_BUS_WIDTH),
612     .ADDR_SERIAL_BIT_WIDTH(ADDR_SERIAL_BIT_WIDTH),
613     .DATA_SERIAL_BIT_WIDTH(DATA_SERIAL_BIT_WIDTH),
614     .SLAVE1_ADDR_RANGE_BEGIN(SLAVE1_ADDR_RANGE_BEGIN),
615     .SLAVE1_ADDR_RANGE_END(SLAVE1_ADDR_RANGE_END),
616     .SLAVE2_ADDR_RANGE_BEGIN(SLAVE2_ADDR_RANGE_BEGIN),
617     .SLAVE2_ADDR_RANGE_END(SLAVE2_ADDR_RANGE_END),
618     .SLAVE3_ADDR_RANGE_BEGIN(SLAVE3_ADDR_RANGE_BEGIN),
619     .SLAVE3_ADDR_RANGE_END(SLAVE3_ADDR_RANGE_END)
620 )
621 top(
622     .clk(clk),
623     .rst_reset(rst_reset),
624     .reset(reset),
625
626     .boss1RequestIn(boss1RequestIn),
627     .boss2RequestIn(boss2RequestIn),
628
629     .boss1AddressIn(boss1AddressIn),
630     .boss2AddressIn(boss2AddressIn),
631
632     .boss1AddressValidIn(boss1AddressValidIn),
633     .boss2AddressValidIn(boss2AddressValidIn),
634
635     .boss1ReadIn(boss1ReadIn),
636     .boss2ReadIn(boss2ReadIn),
637
638     .boss1WriteIn(boss1WriteIn),
639     .boss2WriteIn(boss2WriteIn),
640
641     .boss1DataIn(boss1DataIn),
642     .boss2DataIn(boss2DataIn),
643
644     .boss1DataValidIn(boss1DataValidIn),
645     .boss2DataValidIn(boss2DataValidIn),
646
647     .slave1DataIn(slave1DataIn),
648     .slave2DataIn(slave2DataIn),

```

```

643     .slave3DataIn (slave3DataIn) ,
644
645     .slave1DataValidIn (slave1DataValidIn) ,
646     .slave2DataValidIn (slave2DataValidIn) ,
647     .slave3DataValidIn (slave3DataValidIn) ,
648
649     .slave1WriteResponseIn (slave1WriteResponseIn) ,
650     .slave2WriteResponseIn (slave2WriteResponseIn) ,
651     .slave3WriteResponseIn (slave3WriteResponseIn) ,
652
653     .slave1BusyIn (slave1BusyIn) ,
654     .slave2BusyIn (slave2BusyIn) ,
655     .slave3BusyIn (slave3BusyIn) ,
656
657     .boss1DataOut (boss1DataOut) ,
658     .boss2DataOut (boss2DataOut) ,
659
660     .boss1DataValidOut (boss1DataValidOut) ,
661     .boss2DataValidOut (boss2DataValidOut) ,
662
663     .slave1DataValidOut (slave1DataValidOut) ,
664     .slave2DataValidOut (slave2DataValidOut) ,
665     .slave3DataValidOut (slave3DataValidOut) ,
666
667     .slave1DataOut (slave1DataOut) ,
668     .slave2DataOut (slave2DataOut) ,
669     .slave3DataOut (slave3DataOut) ,
670
671     .slave1AddrValidOut (slave1AddrValidOut) ,
672     .slave2AddrValidOut (slave2AddrValidOut) ,
673     .slave3AddrValidOut (slave3AddrValidOut) ,
674
675     .slave1AddrOut (slave1AddrOut) ,
676     .slave2AddrOut (slave2AddrOut) ,
677     .slave3AddrOut (slave3AddrOut) ,
678
679     .slave1ReadEnOut (slave1ReadEnOut) ,
680     .slave2ReadEnOut (slave2ReadEnOut) ,
681     .slave3ReadEnOut (slave3ReadEnOut) ,
682
683     .slave1WriteEnOut (slave1WriteEnOut) ,
684     .slave2WriteEnOut (slave2WriteEnOut) ,
685     .slave3WriteEnOut (slave3WriteEnOut) ,
686
687     .slave1DataReadyOut (slave1DataReadyOut) ,
688     .slave2DataReadyOut (slave2DataReadyOut) ,
689     .slave3DataReadyOut (slave3DataReadyOut) ,
690
691     .boss1DataAddrReadyOut (boss1DataAddrReadyOut) ,
692     .boss2DataAddrReadyOut (boss2DataAddrReadyOut) ,
693
694     .boss1NoAddressOut (boss1NoAddressOut) ,
695     .boss2NoAddressOut (boss2NoAddressOut) ,
696
697     .boss1TransIncompleteOut (boss1TransIncompleteOut) ,
698     .boss2TransIncompleteOut (boss2TransIncompleteOut)
699 );
700
701 endmodule
702

```

```

1  `timescale 1ns / 1ps
2  //////////////////////////////////////////////////
3  // Company: ENTC, University of Moratuwa
4  // Author: Miyuru Thathsara
5  //
6  // Create Date: 12/18/2020 10:00:04 AM
7  // Design Name: Master Interface TestBench
8  // Module Name: addrDecodeTB
9  // Project Name: System Bus
10 // Additional Comments:
11 //
12 //////////////////////////////////////////////////
13
14
15 module MasterTb ();
16
17
18 //////////////////////////////////////////////////
19 // Parameters
20 //////////////////////////////////////////////////
21 parameter STATE_REG_WIDTH = 3;
22 parameter STATE_REG_WIDTH_ADDRDECODE = 2;
23 parameter ADDR_SERIAL_BIT_WIDTH = 8;
24 parameter ADDR_COUNTER_WIDTH = 4;
25 parameter DATA_SERIAL_BIT_WIDTH = 8;
26 parameter DATA_COUNTER_WIDTH = 4;
27
28 //////////////////////////////////////////////////
29 // Local Parameters
30 //////////////////////////////////////////////////
31 localparam RAND_NUM_BOUND_ADDR = 100; //stick
32 this to more than 2000 to check for address space
33 localparam RAND_NUM_WIDTH_ADDR =
34 ADDR_SERIAL_BIT_WIDTH - 3;
35 localparam RAND_NUM_BOUND_DATA = 100; //stick
36 this to more than 2000 to check for address space
37 localparam RAND_NUM_WIDTH_DATA =
38 DATA_SERIAL_BIT_WIDTH - 1;
39 localparam RAND_NUM_WIDTH =
40 DATA_SERIAL_BIT_WIDTH;
41
42
43 //////////////////////////////////////////////////
44 //LOOP PARAM//
45 //For loop integers
46 integer i1;
47 integer i2;
48 integer i3;
49
50
51 reg start_bit = 1'b0;
52 reg rd_bit = 1'b0;
53 reg wr_bit = 1'b0;
54
55 reg [RAND_NUM_WIDTH_ADDR-1:0] rand_addr;
56 reg [RAND_NUM_WIDTH_DATA-1:0] rand_data;
57 reg [RAND_NUM_WIDTH_DATA-1:0] rand_data_two;
58
59 reg [3-1:0] rand_junction;
60 reg [ADDR_SERIAL_BIT_WIDTH-1:0] addr_integrate;
61 reg [DATA_SERIAL_BIT_WIDTH-1:0] data_integrate;
62 reg [DATA_SERIAL_BIT_WIDTH-1:0] data_integrate_two;
63
64 reg [ADDR_SERIAL_BIT_WIDTH-1:0] tempAddr;
65 reg [DATA_SERIAL_BIT_WIDTH-1:0] tempDATA;
66
67 //////////////////////////////////////////////////
68 // Registers and Wires
69 //////////////////////////////////////////////////

```

```

65 reg reset;
66
67 reg BossRequestIn; // Boss
68 Makes a request here using this. It pulls it up.
69 //reg grantAccessIn;
70
71 reg BossRequestIn2; // Boss
72 Makes a request here using this. It pulls it up.
73 //reg grantAccessIn2;
74
75 reg [ADDR_SERIAL_BIT_WIDTH-1:0] BossAddressIn; //Boss
76 loads the 'Address' of the Slave here. it is of the format (1)(01)(00000) ->
77 (Activate) (R) (W) (Data-5bit)
78 reg [DATA_SERIAL_BIT_WIDTH-1:0] BossDATAIn; //If the
79 Boss wants a 'Write' operation done, it will send the data in here.
80
81 reg BossAddressIn2; //Boss
82 loads the 'Address' of the Slave here. it is of the format (1)(01)(00000) ->
83 (Activate) (R) (W) (Data-5bit)
84 reg [DATA_SERIAL_BIT_WIDTH-1:0] BossDATAIn2; //If the
85 Boss wants a 'Write' operation done, it will send the data in here.
86
87 reg preSwapIn;
88
89 wire releasedIn; //Message
90 from Arbiter indicating the master has been released.
91
92
93 wire requestOut;
94 wire requestOut2;
95
96 wire [ 1 : 0 ] case_reg_arbiter;
97 wire [ 1 : 0 ] grantAccessOut;
98 wire [STATE_REG_WIDTH-1:0] case_reg;
99 wire [STATE_REG_WIDTH-1:0] case_reg2;
100
101 wire [ADDR_SERIAL_BIT_WIDTH-1:0] AddrSHIFT;
102 wire [ADDR_SERIAL_BIT_WIDTH-1:0] AddrSHIFT2;
103
104 reg splitIn2;
105 reg splitIn;
106
107 wire DATAOUT;
108 wire [DATA_SERIAL_BIT_WIDTH-1:0] DATA;
109 wire MasterSplitOut = 0;
110 //(You are indicating the bus that you are in a SPLIT state)
111
112 reg preSwapIn2;
113
114 wire DATAOUT2;
115 wire [DATA_SERIAL_BIT_WIDTH-1:0] DATA2;
116 wire MasterSplitOut2 = 0;
117 //(You are indicating the bus that you are in a SPLIT state)
118
119 ///////////////////////////////////////////////////
120 //Create a clock
121 ///////////////////////////////////////////////////
122 reg clk = 1;
123 localparam period = 20;
124 always #10 clk <= ~clk;
125
126 integer f;
127
128 reg Start;
129 reg load;
130 reg [7:0] LOAD;
131 reg done;
132 wire [7:0] SLAVEDATA;

```

[illegible]

```

192 //////////////////////////////////////////////////
193
194 parameter ADDR_RANGE_BEGIN_ONE           = 0;
195 parameter ADDR_RANGE_END_ONE             = 16;
196
197 parameter ADDR_RANGE_BEGIN_TWO           = 16;
198 parameter ADDR_RANGE_END_TWO             = 25;
199
200 parameter ADDR_RANGE_BEGIN_THREE         = 25;
201 parameter ADDR_RANGE_END_THREE           = 32;
202
203
204 reg SplitIn1;
205 reg SplitIn2;
206 reg SplitIn3;
207
208
209 reg D1;
210 reg D2;
211 reg D3;
212
213 wire [ STATE_REG_WIDTH_ADDRDECODE - 1 : 0 ] State_one;
214 wire [ STATE_REG_WIDTH_ADDRDECODE - 1 : 0 ] State_two;
215 wire [ STATE_REG_WIDTH_ADDRDECODE - 1 : 0 ] State_three;
216
217 wire [ ADDR_SERIAL_BIT_WIDTH - 1 : 0 ] Valid_addr_1;
218 wire [ ADDR_SERIAL_BIT_WIDTH - 1 : 0 ] Valid_addr_2;
219 wire [ ADDR_SERIAL_BIT_WIDTH - 1 : 0 ] Valid_addr_3;
220
221 wire Ack_one;
222 wire Ack_two;
223 wire Ack_three;
224
225 wire Ready_one;
226 wire Ready_two;
227 wire Ready_three;
228
229 //////////////////////////////////////////////////
230 ///ADDRESS MULTIPLEXOR
231 wire AddressLine;
232 assign AddressLine = (grantAccessOut[0]|grantAccessOut[1])?(grantAccessOut[1] ?
  (AddrSHIFT[7]): (1'b0) ):(grantAccessOut[0] ? (AddrSHIFT2[7]): (1'b0) );
233 //////////////////////////////////////////////////
234
235 wire [7:0] SR1;
236 wire [7:0] SR2;
237 wire [7:0] SR3;
238
239 addrDecode #(
240     .STATE_REG_WIDTH(STATE_REG_WIDTH_ADDRDECODE),
241     .ADDR_SERIAL_BIT_WIDTH(ADDR_SERIAL_BIT_WIDTH),
242     .ADDR_RANGE_BEGIN(ADDR_RANGE_BEGIN_ONE),
243     .ADDR_RANGE_END(ADDR_RANGE_END_ONE)
244 )SlaveOne(
245     .clk(clk),
246     .reset(reset),
247     .addrIn(AddressLine),
248     .splitIn(SplitIn1),
249     .addrValidOut(Valid_addr_1),
250     .stateRegOut(State_one),
251     .ackSignalOut(Ack_one),
252     .readyOut(Ready_one),
253     .DoneIn(D1),
254     .shift_reg(SR1)
255 );
256
257
258 addrDecode #(

```

```

259     .STATE_REG_WIDTH (STATE_REG_WIDTH_ADDRDECODE),
260     .ADDR_SERIAL_BIT_WIDTH (ADDR_SERIAL_BIT_WIDTH),
261     .ADDR_RANGE_BEGIN (ADDR_RANGE_BEGIN_TWO),
262     .ADDR_RANGE_END (ADDR_RANGE_END_TWO)
263 ) SlaveTwo (
264     .clk (clk),
265     .reset (reset),
266     .addrIn (AddressLine),
267     .splitIn (SplitIn2),
268     .addrValidOut (Valid_addr_2),
269     .stateRegOut (State_two),
270     .ackSignalOut (Ack_two),
271     .readyOut (Ready_two),
272     .DoneIn (D2),
273     .shift_reg (SR2)
274 );
275
276
277 addrDecode #(
278     .STATE_REG_WIDTH (STATE_REG_WIDTH_ADDRDECODE),
279     .ADDR_SERIAL_BIT_WIDTH (ADDR_SERIAL_BIT_WIDTH),
280     .ADDR_RANGE_BEGIN (ADDR_RANGE_BEGIN_THREE),
281     .ADDR_RANGE_END (ADDR_RANGE_END_THREE)
282 ) SlaveThree (
283     .clk (clk),
284     .reset (reset),
285     .addrIn (AddressLine),
286     .splitIn (SplitIn3),
287     .addrValidOut (Valid_addr_3),
288     .stateRegOut (State_three),
289     .ackSignalOut (Ack_three),
290     .readyOut (Ready_three),
291     .DoneIn (D3),
292     .shift_reg (SR3)
293
294 );
295
296
297 ///////////////////////////////////////////////////////////////////
298 ///////////////////////////////////////////////////////////////////ARBITER
299 //dismissedIn (extracted out)
300
301 busArbitor BA(
302     .clk (clk),
303     .reset (reset),
304     .requestIn ({requestOut, requestOut2}),
305     .splitIn (splitIn_arb),
306     .dismissedIn (endOut),
307     .unsplitIn (unsplitIn),
308     .grantAccessOut (grantAccessOut),
309     .releasedIn (releasedIn),
310     .Master_split_stateIn ({MasterSplitOut, MasterSplitOut2}),
311     .case_reg (case_reg_arbiter)
312 );
313
314
315
316
317
318
319
320
321
322 // Bounded Random Number generator function
323 function [ RAND_NUM_WIDTH - 1 : 0 ] randomBoundNum;
324     input integer bound;
325     begin
326         randomBoundNum = $urandom % bound;
327     end

```

```

328 endfunction
329
330
331
332 initial begin
333     unsplitIn = 0;
334     rand_addr      <= { RAND_NUM_WIDTH_ADDR {1'b0} };
335     rand_data       <= { RAND_NUM_WIDTH_DATA {1'b0} };
336     rand_data_two   <= { RAND_NUM_WIDTH_DATA {1'b0} };
337
338     SplitIn1 = 0;
339     SplitIn2 = 0;
340     SplitIn3 = 0;
341
342     D1 = 0;
343     D2 = 0;
344     D3 = 0;
345
346     f = $fopen("output.txt","w");
347
348     //grantAccessIn = 0;
349     BossRequestIn   = 0;
350
351     Start=0;
352     load=0;
353     done =0;
354
355     preSwapIn = 0;
356     splitIn =0;
357
358     #period;
359     reset = 0;
360     #period;
361     reset = 1;
362     #(2*period);
363     reset = 0;
364     #period;
365
366     for(i1 = 0; i1 < 100; i1 = i1 + 1)begin
367
368         $display("Iteration: ", i1);
369         $fdisplay(f, "/n");
370         $fdisplay(f, i1);
371
372         //Creating the address data for the second master
373         start_bit      = 1'b1;
374         rd_bit          = randomBoundNum(2);
375         wr_bit          = ~rd_bit;
376         rand_addr       = randomBoundNum(RAND_NUM_BOUND_ADDR);
377         addr_integrate  = { start_bit, rd_bit, wr_bit, rand_addr };
378
379         //address data for the second master
380         //rd_bit_2       = randomBoundNum(2);
381         //rand_addr_2    = randomBoundNum(RAND_NUM_BOUND_ADDR);
382         //addr_integrate_2 = { start_bit, rd_bit_2, ~rd_bit_2,
383                             rand_addr_2 };
384
385         $display("Random Number: ", addr_integrate);
386         $fdisplay(f, "%8b", addr_integrate);
387
388         //Creating data for write or read
389         start_bit      = 1'b1;
390         rand_data       = randomBoundNum(RAND_NUM_BOUND_ADDR);
391         rand_data_two   = randomBoundNum(RAND_NUM_BOUND_ADDR);
392         data_integrate  = { start_bit, rand_data };
393         data_integrate_two = { start_bit, rand_data_two };
394
395         $display("Random Number: ", data_integrate);
396         $fdisplay(f, "%8b", data_integrate);

```

```

396
397
398     BossAddressIn    = addr_integrate;
399
400     if (wr_bit) begin
401         BossDATAIn = data_integrate;
402         LOAD = data_integrate_two;
403         load =1;
404     end
405     else begin
406         BossDATAIn    = data_integrate;
407         LOAD =        data_integrate_two;
408         load =1;
409     end
410     #(2*period);
411     load = 0;
412
413
414     BossRequestIn    = 1;
415
416
417
418     ////////////////////////////////////////////
419     //
420     #(3*period); //Waiting to see if Master requests for data. If so it will grant
421     access.
422
423     if (grantAccessOut[1]) begin
424         //grantAccessIn =1;
425         #period;
426         for(i2 = 0; i2 < ADDR_SERIAL_BIT_WIDTH; i2 = i2 + 1)begin
427             #period;
428             end
429             BossRequestIn =0;
430         end
431
432         #period;
433         #period;
434         //Check if the Address was unloaded successfully.
435         if ( (addr_integrate[ ADDR_SERIAL_BIT_WIDTH - 4 : 0 ] >= ADDR_RANGE_BEGIN_ONE)
436         && (addr_integrate[ ADDR_SERIAL_BIT_WIDTH - 4 : 0 ] < ADDR_RANGE_END_ONE) ) begin
437             $display("Slave 01");
438             if ((Ack_one) && (Valid_addr_1==addr_integrate)) begin
439                 $display("Address Transfer Sucess");
440
441             end else begin
442                 $display("Address Transfer Fail");
443
444             end
445         end else if ( (addr_integrate[ ADDR_SERIAL_BIT_WIDTH - 4 : 0 ] >=
446         ADDR_RANGE_BEGIN_TWO) && (addr_integrate[ ADDR_SERIAL_BIT_WIDTH - 4 : 0 ] <
447         ADDR_RANGE_END_TWO) ) begin
448             $display("Slave 02");
449             if ((Ack_two) && (Valid_addr_2==addr_integrate)) begin
450                 $display("Address Transfer Sucess");
451
452             end else begin
453                 $display("Address Transfer Fail");
454
455             end
456         end
457     else
458         begin
459             $display("Slave 03");
460             if ((Ack_three) && (Valid_addr_3==addr_integrate)) begin
461                 $display("Address Transfer Sucess");
462
463             end else begin
464                 $display("Address Transfer Fail");

```

```

459
460         end
461     end
462
463
464
465
466
467     //Now we are in the junction room
468     //Split Check...Release Check....Preswaping
469     #(period*3)
470
471     preSwapIn = 1;
472     Start =1;
473     //Part where we do the swapping.
474     for(i3 = 0; i3 < DATA_SERIAL_BIT_WIDTH; i3 = i3 + 1)begin
475         #(period);
476     end
477
478     //#(period*DATA_SERIAL_BIT_WIDTH);
479     Start =0;
480     #(period*(1.5))
481
482     //Check if the data was unloaded successfully.
483     if (wr_bit) begin
484         if (SLAVEDATA==data_integrate) begin
485             if (DATA==data_integrate_two) begin
486                 $display("Write Success");
487             end else begin
488                 $display("Write Fail, DATA");
489             end
490         end else begin
491             if (DATA==data_integrate_two) begin
492                 $display("Write Fail, SLAVE");
493             end else begin
494                 $display("Write Fail, DATA,SLAVE");
495             end
496         end
497     end else begin
498         if (SLAVEDATA==data_integrate) begin
499             if (DATA==data_integrate_two) begin
500                 $display("Read Success");
501             end else begin
502                 $display("Read Fail, DATA");
503             end
504         end else begin
505             if (DATA==data_integrate_two) begin
506                 $display("Read Fail, SLAVE");
507             end else begin
508                 $display("Read Fail, DATA,SLAVE");
509             end
510         end
511     end
512     #(period*(0.5))
513     D1 = 1;
514     D2 = 1;
515     D3 = 1;
516
517     #period;
518     #period;
519     D1 = 0;
520     D2 = 0;
521     D3 = 0;
522     //grantAccessIn =0;
523     preSwapIn = 0;
524     tempAddr = 0;
525     #(2*period);
526     done = 1;
527     #(2*period);

```

```
528
529     end
530     $fclose(f);
531 end
532 endmodule
```

```

1  `timescale 1ns / 1ps
2  //////////////////////////////////////////////////
3  // Company: ENTC, University of Moratuwa
4  // Author: Miyuru Thathsara
5  //
6  // Create Date: 12/20/2020 11:46:20 AM
7  // Design Name: Slave Interface TestBench
8  // Module Name: slaveInterfaceTB
9  // Project Name: System Bus
10 // Additional Comments:
11 //
12 //////////////////////////////////////////////////
13
14
15 module slaveInterfaceTB();
16
17 //////////////////////////////////////////////////
18 // Parameters
19 //////////////////////////////////////////////////
20 parameter DATA_BUS_WIDTH = 16;
21 parameter ADDR_BUS_WIDTH = 16;
22 parameter ADDR_SERIAL_BIT_WIDTH = 19;
23 //////////////////////////////////////////////////
24 // Local Parameters
25 //////////////////////////////////////////////////
26 localparam ADDR_BOUND = 100;
27 localparam DATA_BOUND = 2000;
28 localparam NUM_OF_ADDRESSES = 10;
29 localparam ADDR_IN_BIT_WIDTH =
ADDR_SERIAL_BIT_WIDTH - 1;
30 //////////////////////////////////////////////////
31 // Wires and Registers
32 //////////////////////////////////////////////////
33 reg clk;
34 reg rst_reset;
35 reg reset;
36 reg addrValidIn;
37 reg [ ADDR_IN_BIT_WIDTH - 1 : 0 ] addrIn;
38 reg busyIn;
39 reg [ DATA_BUS_WIDTH - 1 : 0 ] slaveDataIn;
40 reg slaveDataValidIn;
41 reg dataIn;
42 reg slaveWriteResponseIn;
43 reg releasedIn;
44 wire addrReadyOut;
45 wire dataOut;
46 wire slaveDataReadyOut;
47 wire slaveAddrValidOut;
48 wire slaveDataValidOut;
49 wire dataReadyOut;
50
51 reg [ ADDR_BUS_WIDTH - 1 : 0 ] addr_reg;
52 reg [ DATA_BUS_WIDTH : 0 ] data_reg;
53 reg wr_bit;
54 reg rd_bit;
55 integer i1;
56 integer i2;
57 //////////////////////////////////////////////////
58 // Implementation
59 //////////////////////////////////////////////////
60
61 slaveInterface#(
62     .DATA_BUS_WIDTH(DATA_BUS_WIDTH),
63     .ADDR_BUS_WIDTH(ADDR_BUS_WIDTH),
64     .ADDR_SERIAL_BIT_WIDTH(ADDR_SERIAL_BIT_WIDTH)
65 )
66 slaveInterface(
67     .clk(clk),
68     .rst_reset(rst_reset),

```

[illegible]

```

133     while(addrReadyOut)begin
134         @(posedge clk);
135     end
136     #10;
137     addrIn = {
138         ADDR_IN_BIT_WIDTH {1'b0} };
139     addrValidIn = 1'b0;
140     if(rd_bit && ~wr_bit)begin
141         $display("Reading Operation!!!! busyIn == ", busyIn);
142         if(busyIn)begin
143             #30;
144             busyIn = 1'b0;
145             #50;
146             releasedIn = 1'b1;
147             #10;
148             releasedIn = 1'b0;
149         end
150         while(~slaveDataReadyOut)begin
151             @(posedge clk);
152         end
153         #10;
154         slaveDataValidIn = 1'b1;
155         slaveDataIn =
156             randomBoundNum(DATA_BOUND);
157         while(slaveDataReadyOut)begin
158             @(posedge clk);
159         end
160         #10;
161         slaveDataValidIn = 1'b0;
162         slaveDataIn = {
163             DATA_BUS_WIDTH {1'b0} };
164     end
165     else begin
166         if(~rd_bit && wr_bit)begin
167             $display("Writing Operation!!!! busyIn == ", busyIn);
168             if(busyIn)begin
169                 #30;
170                 busyIn = 1'b0;
171                 #50;
172                 releasedIn = 1'b1;
173                 #10;
174                 releasedIn = 1'b0;
175             end
176             while(~dataReadyOut)begin
177                 @(posedge clk);
178             end
179             #10;
180             data_reg = { 1'b1,
181                 randomBoundNum(DATA_BOUND) };
182             dataIn = data_reg[
183                 DATA_BUS_WIDTH ];
184             for(i2 = 0; i2 < DATA_BUS_WIDTH; i2 = i2 + 1)begin
185                 #10;
186                 data_reg = data_reg << 1;
187                 dataIn = data_reg[
188                     DATA_BUS_WIDTH ];
189             end
190             while(~slaveDataValidOut && ~slaveAddrValidOut)begin
191                 @(posedge clk);
192             end
193             #10;
194             slaveWriteResponseIn = 1'b1;
195             #10;
196             slaveWriteResponseIn = 1'b0;
197             dataIn = {
198                 (DATA_BUS_WIDTH + 1) {1'b0} };
199         end
200     end
201 end

```

```
195         end
196
197     endmodule
198
```

```

1  `timescale 1ns / 1ps
2  //////////////////////////////////////////////////
3  // Company: ENTC, University of Moratuwa
4  // Author: Miyuru Thathsara
5  //
6  // Create Date: 12/18/2020 10:00:04 AM
7  // Design Name: Address Decoder TestBench
8  // Module Name: addrDecodeTB
9  // Project Name: System Bus
10 // Additional Comments:
11 //
12 //////////////////////////////////////////////////
13
14
15 module addrDecodeTB();
16
17 //////////////////////////////////////////////////
18 // Parameters
19 //////////////////////////////////////////////////
20 parameter STATE_REG_WIDTH = 2;
21 parameter ADDR_SERIAL_BIT_WIDTH = 19;
22 parameter ADDR_RANGE_BEGIN = 0;
23 parameter ADDR_RANGE_END = 2**10;
24 //////////////////////////////////////////////////
25 // Local Parameters
26 //////////////////////////////////////////////////
27 localparam RAND_NUM_BOUND = 100; //stick this to
more than 2000 to check for address space
28 localparam RAND_NUM_WIDTH = ADDR_SERIAL_BIT_WIDTH
- 3;
29 //////////////////////////////////////////////////
30 // Registers and Wires
31 //////////////////////////////////////////////////
32 reg clk;
33 reg rst_reset;
34 reg reset;
35 reg [ RAND_NUM_WIDTH - 1 : 0 ] rand;
36 reg addrIn;
37 reg splitIn;
38 wire addrOut;
39 wire addrValidOut;
40 wire [ STATE_REG_WIDTH - 1 : 0 ] stateRegOut;
41 wire ackSignalOut;
42 wire readyOut;
43 reg [ ADDR_SERIAL_BIT_WIDTH - 1 : 0 ] addr_integrate;
44 reg start_bit;
45 reg rd_bit;
46 reg wr_bit;
47 integer i1;
48 integer i2;
49 //////////////////////////////////////////////////
50 // Implementation
51 //////////////////////////////////////////////////
52
53 addrDecode#(
54     .STATE_REG_WIDTH(STATE_REG_WIDTH),
55     .ADDR_SERIAL_BIT_WIDTH(ADDR_SERIAL_BIT_WIDTH),
56     .ADDR_RANGE_BEGIN(ADDR_RANGE_BEGIN),
57     .ADDR_RANGE_END(ADDR_RANGE_END)
58 )
59 addrDecode(
60     .clk(clk),
61     .rst_reset(rst_reset),
62     .reset(reset),
63     .addrIn(addrIn),
64     .splitIn(splitIn),
65     .addrOut(addrOut),
66     .addrValidOut(addrValidOut),
67     .stateRegOut(stateRegOut),

```

```

68     .ackSignalOut(ackSignalOut),
69     .readyOut(readyOut)
70 );
71
72 // Bounded Random Number generator function
73 function [ RAND_NUM_WIDTH - 1 : 0 ] randomBoundNum;
74     input integer bound;
75     begin
76         randomBoundNum = $urandom % bound;
77     end
78 endfunction
79
80 always #5 clk                                     <= ~clk;
81
82 initial begin
83     #0;
84     clk                                     = 1'b0;
85     addrIn                                 = 1'b0;
86     rand                                   = { RAND_NUM_WIDTH {1'b0} };
87     #10;
88     rst_reset                             = 1'b1;
89     reset                                  = 1'b1;
90     #10;
91     rst_reset                             = 1'b0;
92     #10;
93     rst_reset                             = 1'b1;
94     #10;
95     reset                                  = 1'b0;
96     #10;
97     reset                                  = 1'b1;
98     start_bit                             = 1'b0;
99     rd_bit                                = 1'b0;
100    wr_bit                                 = 1'b0;
101    for(i1 = 0; i1 < 10; i1 = i1 + 1)begin
102        #250;
103        if(readyOut)begin
104            rand                             = randomBoundNum(RAND_NUM_BOUND);
105            start_bit                         = 1'b1;
106            rd_bit                           = randomBoundNum(2);
107            wr_bit                           = ~rd_bit;
108            splitIn                          = randomBoundNum(2);
109            addr_integrate                    = { start_bit, rd_bit, wr_bit, rand };
110            $display("Random Number: ", rand);
111            for(i2 = 0; i2 < ADDR_SERIAL_BIT_WIDTH; i2 = i2 + 1)begin
112                #10;
113                addrIn                        = addr_integrate[ ADDR_SERIAL_BIT_WIDTH - 1
114                addr_integrate                = addr_integrate << 1;
115            end
116            #10;
117            addrIn                            = 1'b0;
118        end
119    end
120 end
121 endmodule
122

```